

Programmazione Competitiva

Antonio Ianniello

Indice

1	Complessità Computazionale	4
1.1	La notazione asintotica	4
1.1.1	Notazione asintotica O e caso peggiore	4
1.1.2	Notazione asintotica Ω e caso migliore	5
1.1.3	Gerarchia delle complessità asintotiche	6
1.1.4	Algebra della Notazione Asintotica	7
1.2	Complessità computazionale delle principali istruzioni	8
1.3	Esercizi di complessità computazionale	13
1.3.1	Esercizio 1	13
1.3.2	Esercizio 2	14
1.3.3	Esercizio 3	15
2	Strutture dati in C++	16
2.1	Vector	16
2.1.1	Iterazione tramite <code>begin()</code> e <code>end()</code>	16
2.1.2	Operazioni principali sui vector	17
2.1.3	Inserimento ed eliminazione in posizioni specifiche	18
2.1.4	Altre operazioni utili	19
2.1.5	Iterazione su un vector	19
2.1.6	Complessità computazionale riassunta	20
2.2	Map	21
2.2.1	Dichiarazione di una map	21
2.2.2	Operazioni principali sulle map	22
2.2.3	Altre operazioni utili	23
2.2.4	Iterazione su una map	23
2.2.5	Complessità computazionale riassunta	24
2.3	Set	25
2.3.1	Dichiarazione di un set	25
2.3.2	Operazioni principali sui set	25
2.3.3	Altre operazioni utili	28
2.3.4	Iterazione su un set	29
2.3.5	Esempio pratico	29
2.3.6	Complessità computazionale riassunta	30
2.4	Multiset	31
2.4.1	Caratteristiche principali	31
2.4.2	Esempio pratico	31
2.4.3	Complessità computazionale riassunta	32
2.5	Stack	33
2.5.1	Dichiarazione di uno stack	33
2.5.2	Operazioni principali sullo stack	34

2.5.3	Iterazione su uno stack	34
2.5.4	Esempio pratico	35
2.5.5	Complessità computazionale riassunta	35
2.6	Queue	36
2.6.1	Dichiarazione di una queue	36
2.6.2	Operazioni principali sulla queue	36
2.6.3	Iterazione su una queue	37
2.6.4	Esempio pratico	38
2.6.5	Complessità computazionale riassunta	38
2.7	Priority Queue	39
2.7.1	Dichiarazione di una priority_queue	39
2.7.2	Operazioni principali sulla priority_queue	40
2.7.3	Iterazione su una priority_queue	40
2.7.4	Esempio pratico	41
2.7.5	Complessità computazionale riassunta	41
2.8	Deque	42
2.8.1	Dichiarazione di una deque	42
2.8.2	Operazioni principali sulla deque	42
2.8.3	Iterazione su una deque	43
2.8.4	Esempio pratico	44
2.8.5	Complessità computazionale riassunta	45
2.8.6	Differenze tra queue e deque	45
3	Brute Force e Algoritmi Greedy	47
3.1	Algoritmi Brute Force	47
3.1.1	Antivirus	48
3.1.2	Stick Lengths	53
3.1.3	Nearest Smaller Values	56
3.2	Idee Greedy	59
3.2.1	Calcolatrice Rotta	60
3.2.2	Sticks Length	61
A	Regolamento 2025/2026	63
B	Pseudocodice	72

Introduzione

I Campionati Italiani di Informatica (ex Olimpiadi) sono una competizione rivolta agli studenti delle istituzioni scolastiche secondarie di II grado. Giunte ormai alla XXVI edizione, fanno parte del programma di valorizzazione delle eccellenze che la Direzione Generale per gli ordinamenti scolastici e la valutazione del sistema nazionale di istruzione del Ministero dell'Istruzione promuove e finanzia ogni anno.

La partecipazione è aperta alle classi I-IV di tutte le istituzioni scolastiche secondarie di II grado, statali e paritarie, che ritengano di avere studenti con potenziale interesse per l'informatica, soprattutto riguardo gli aspetti logici e algoritmici di tale disciplina. La partecipazione è adatta e consigliata anche a coloro che ancora non hanno studiato informatica a livello curricolare, per consentire agli studenti di interessarsi e avvicinarsi gradualmente alla materia.

Gli studenti potranno prepararsi alla prima fase tramite le prove degli anni passati disponibili sul sito `scolastiche.olinfo.it`.

Il regolamento, inclusa la descrizione delle varie fasi è riportato in appendice A.

Si fa notare che durante la prima fase (selezione scolastica), verrà richiesta la soluzione dei problemi utilizzando esclusivamente **pseudocodice** le cui regole sono riportate in appendice B

In questo documento verranno trattati argomenti teorici e pratici al fine di affrontare le olimpiadi di informatica con una degna preparazione.

Capitolo 1

Complessità Computazionale

La programmazione competitiva è un'attività di programmazione finalizzata alla partecipazione a gare organizzate in cui i partecipanti devono risolvere problemi algoritmici e matematici scrivendo programmi che soddisfano specifiche restrizioni di tempo e memoria.

È importante valutare la complessità di ogni algoritmo costruito in quanto può essere necessario limitare l'esecuzione ad un certo tempo (es. l'algoritmo deve terminare entro 2 secondi dallo start).

Nel contesto della competizione, consideriamo un algoritmo eseguito da un computer in grado di effettuare circa 10^8 operazioni al secondo. Sarà sufficiente calcolare il numero di operazioni richieste dall'algoritmo per determinare il tempo totale necessario per completarne l'esecuzione.

Ovviamente, calcolare il numero totale di operazioni potrebbe non essere semplice, poiché esso dipende sia dal numero di dati in input sia dall'efficienza "interna" degli algoritmi (che approfondiremo più avanti).

1.1 La notazione asintotica

Per valutare la complessità temporale di un algoritmo, occorre introdurre un concetto matematico importantissimo: **la notazione asintotica**.

In matematica, questa consente di confrontare il tasso di crescita (comportamento asintotico) di una funzione nei confronti di un'altra.

In informatica si sfrutta questo concetto per valutare di quanto aumenta il tempo di esecuzione di un algoritmo ($T(n)$) in relazione all'aumento dei dati in input (n).

Esistono 3 tipi di notazione asintotica:

- **Notazione asintotica O** , chiamata *limite superiore asintotico*
- **Notazione asintotica Ω** , chiamata *limite inferiore asintotico*
- **Notazione asintotica Θ** , chiamata *limite stretto asintotico*

Andiamo nel dettaglio di ognuna per capire come possono esserci utili

1.1.1 Notazione asintotica O e caso peggiore

Date due funzioni $T(n)$ e $f(n)$, si dice che $T(n) \in O(f(n))$ se esistono costanti positive c e n_0 tali che:

$$0 \leq T(n) \leq c \cdot f(n) \quad \text{per ogni } n \geq n_0$$

In altre parole, a partire da un certo valore di n , $f(n)$ rappresenta un limite superiore asintotico per la crescita di $T(n)$.

Informalmente, $T(n) \in O(f(n))$ se $T(n)$ **non cresce più velocemente di** $f(n)$. Approssimando la funzione $T(n)$ con $f(n)$ per $n > n_0$, si ottiene una **sovrastima** di $T(n)$.

In informatica, quando analizziamo un algoritmo, ci interessa generalmente il **caso peggiore**. Il tempo reale di esecuzione può variare a seconda dell'input, e calcolarlo esattamente per tutti i possibili input è spesso **molto difficile** o impossibile. Per questo motivo, sostituendo $T(n)$ con una funzione più semplice come $c \cdot f(n)$, che **sovrastima** il tempo reale, otteniamo una stima della complessità temporale valida in **ogni situazione possibile**.

In figura 1.1 è riportata la rappresentazione grafica di quanto detto. Sostituendo $T(n)$ con $c \cdot f(n)$, stiamo considerando il **caso peggiore** possibile, senza dover calcolare ogni possibile esecuzione dell'algoritmo.

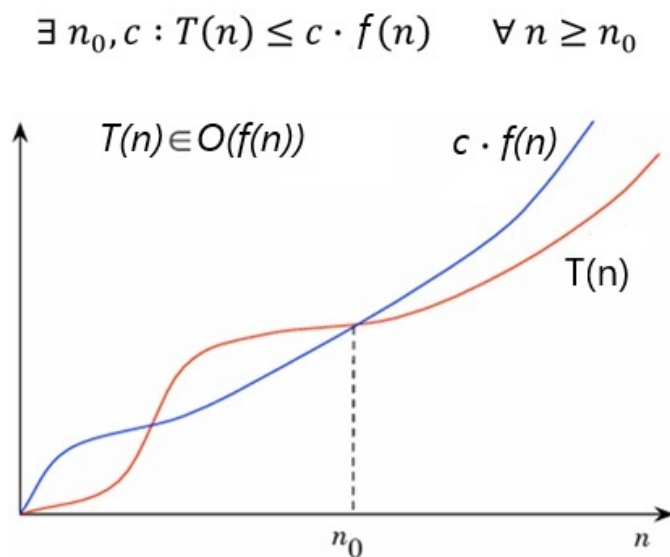


Figura 1.1: Rappresentazione grafica della notazione O e del caso peggiore

1.1.2 Notazione asintotica Ω e caso migliore

Date due funzioni $T(n)$ e $f(n)$, si dice che $T(n) \in \Omega(f(n))$ se esistono costanti positive c e n_0 tali che:

$$0 \leq c \cdot f(n) \leq T(n) \quad \text{per ogni } n \geq n_0$$

In altre parole, a partire da un certo valore di n , $f(n)$ rappresenta un limite inferiore asintotico per la crescita di $T(n)$.

Informalmente, $T(n) \in \Omega(f(n))$ se $T(n)$ **non cresce più lentamente di** $f(n)$. Approssimando la funzione $T(n)$ con $f(n)$ per $n > n_0$, si ottiene una **sottostima** di $T(n)$.

In informatica, il limite inferiore viene utilizzato per indicare il **caso migliore** di esecuzione di un algoritmo. Il tempo reale di esecuzione può variare a seconda dell'input, ma sostituendo $T(n)$ con una funzione semplice come $c \cdot f(n)$ che **sottostima** il tempo reale, otteniamo una stima valida almeno quanto il tempo minimo richiesto dall'algoritmo.

In figura 1.2 è riportata la rappresentazione grafica di quanto detto. Sostituendo $T(n)$ con $c \cdot f(n)$ in questo contesto, stiamo considerando il **caso migliore** possibile.

$$\exists n_0, c : T(n) \geq c \cdot f(n) \quad \forall n \geq n_0$$

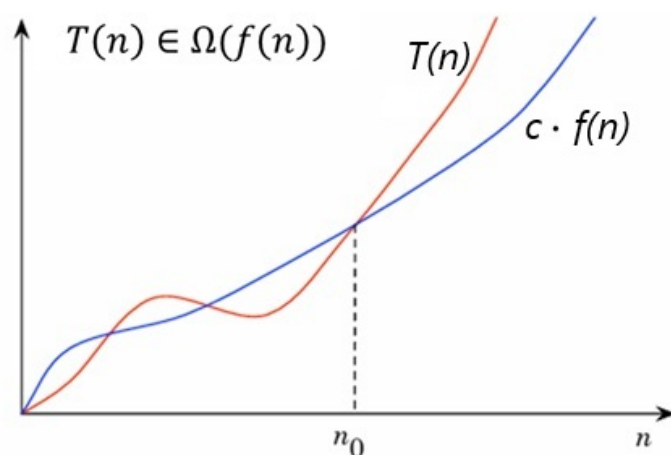


Figura 1.2: Rappresentazione grafica della notazione Ω e del caso migliore

1.1.3 Gerarchia delle complessità asintotiche

Per avere un'idea chiara della velocità di crescita delle principali funzioni, possiamo organizzare le complessità asintotiche in una tabella. La tabella 1.1 mostra le funzioni più comuni ordinate in **ordine crescente di crescita**, partendo dalle più lente fino a quelle che crescono più rapidamente.

Ordine di crescita	Funzioni tipiche
Costante	$O(1)$
Logaritmica	$O(\log n)$
Radicale	$O(\sqrt{n})$
Lineare	$O(n)$
Lineare-logaritmica	$O(n \log n)$
Quadratica	$O(n^2)$
Cubica	$O(n^3)$
Polinomiale	$O(n^k), k > 3$
Esponenziale	$O(2^n), O(k^n)$
Fattoriale	$O(n!)$
Super-esponenziale	$O(n^n)$

Tabella 1.1: Funzioni comuni ordinate per crescita asintotica

1.1.4 Algebra della Notazione Asintotica

Per semplificare il calcolo del costo computazionale asintotico degli algoritmi si possono sfruttare delle semplici regole che enunciamo senza riportarne la dimostrazione¹.

Prima regola: moltiplicazione per costante

Se esiste una funzione $T(n) \geq 0$ tale che:

$$T(n) \in O(f(n)),$$

allora, per ogni costante positiva $k > 0$, vale:

$$k \cdot T(n) \in O(f(n)).$$

Informalmente, possiamo enunciare questa regola dicendo che: **le costanti moltiplicative si possono ignorare.**

Seconda regola: somma

Siano $T(n) \geq 0$ e $S(n) \geq 0$ due funzioni tali che:

$$T(n) \in O(f(n)) \quad \text{e} \quad S(n) \in O(g(n)).$$

Allora vale:

$$T(n) + S(n) \in O(f(n) + g(n)) = O(\max(f(n), g(n))).$$

In altre parole:

Se $f(n)$ cresce più velocemente di $g(n)$, allora $O(f(n) + g(n)) = O(f(n))$.

Se $g(n)$ cresce più velocemente di $f(n)$, allora $O(f(n) + g(n)) = O(g(n))$.

In termini ancora più semplici, possiamo affermare che in una somma si può **trascurare** il termine più piccolo.

Terza regola: prodotto

Siano $T(n) \geq 0$ e $S(n) \geq 0$ due funzioni tali che:

$$T(n) \in O(f(n)) \quad \text{e} \quad S(n) \in O(g(n)).$$

Allora vale:

$$T(n) \cdot S(n) \in O(f(n) \cdot g(n)).$$

MOLTO informalmente, possiamo scrivere che:

$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$$

¹Le regole che verranno riportate sono valide sia per il caso peggiore che per il caso migliore. Visto che nei problemi informatici siamo interessati quasi sempre al caso peggiore, riporteremo l'algebra della notazione asintotica **O**

1.2 Complessità computazionale delle principali istruzioni

Operazioni elementari

Tutte le operazioni elementari, ovvero:

- assegnazione di una variabile,
- somma, sottrazione, moltiplicazione, divisione,
- confronto ($<$, $>$, $==$, $!=$, ...),
- accesso all'elemento di un array con indice noto,
- input o output di una singola variabile,

hanno sempre complessità computazionale $O(1)$.

Consideriamo il seguente frammento di codice C++:

```
1  int a = 5;
2  int b = 10;
3  int c;
4  int d;
5  int e;
6
7  if (a < b) {
8      c = a + b;
9      d = c * 2;
10 }
11 else {
12     c = a - b;
13     d = c * 3;
14     e = d + 1;
15 }
```

Analizziamo il costo computazionale passo per passo:

- `int a = 5;` $\rightarrow O(1)$
- `int b = 10;` $\rightarrow O(1)$
- `if (a < b)` $\rightarrow O(1)$ (un confronto tra due variabili)

A questo punto, il programma può entrare nel ramo `if` oppure nel ramo `else`:

- Ramo `if`: due operazioni elementari $\Rightarrow 2 \cdot O(1) = O(1)$
- Ramo `else`: tre operazioni elementari $\Rightarrow 3 \cdot O(1) = O(1)$

Poiché i due rami sono alternativi (solo uno viene eseguito), si considera il ramo con complessità maggiore, in questo caso il ramo `else`.

La complessità totale del frammento è quindi:

$$O(1) + O(1) + O(1) + O(1) = O(1)$$

In conclusione, l'intero blocco ha complessità **costante**, indipendentemente dal valore delle variabili: il numero di operazioni non dipende da nessun parametro in ingresso.

Complessità computazionale dei cicli

Ciclo for

Consideriamo il seguente frammento di codice C++:

```
1  for (int i = 0; i < n; i++) {  
2      somma += i;  
3  }
```

Analizziamo il comportamento del ciclo:

- L'inizializzazione `int i = 0` ha costo $O(1)$.
- La condizione `i < n` viene valutata $n + 1$ volte $\Rightarrow O(n + 1) = O(n)$.
- L'incremento `i++` viene eseguito n volte $\Rightarrow O(n)$.
- Il corpo del ciclo `somma += i;` ha costo $O(1)$ per iterazione, quindi n iterazioni $\Rightarrow O(n)$.

Sommando i costi:

$$O(1) + O(n) + O(n) + O(n) = O(n)$$

Il ciclo `for` ha dunque complessità complessiva $O(n)$, ovvero cresce linearmente con il numero di iterazioni.

Ciclo while

Il ciclo `while` ha un comportamento simile, ma il numero di iterazioni può dipendere da una condizione variabile. Esempio:

```
1  int i = 1;  
2  while (i < n) {  
3      i = i * 2;  
4  }
```

In questo caso, la variabile `i` raddoppia ad ogni iterazione, quindi il numero di iterazioni totali è pari al numero di volte in cui possiamo moltiplicare per 2 prima di superare n .

$$i = 1 \Rightarrow 2 \Rightarrow 4 \Rightarrow 8 \Rightarrow \dots \Rightarrow n$$

Il ciclo termina quando $2^k \geq n$, quindi $k = \log_2 n$. Di conseguenza, la complessità complessiva è:

$$O(\log n)$$

In generale, per un ciclo `while`, la complessità dipende dal numero di iterazioni necessarie affinché la condizione diventi falsa.

Costrutti Misti e/o Annidati

Quando i cicli sono combinati o condizionati tra loro, la complessità si calcola sommando o moltiplicando i costi dei vari blocchi, a seconda della struttura.

Esempio 1 — Cicli sequenziali:

```
1  for (int i = 0; i < n; i++) {  
2      // O(1)  
3  }  
4  for (int j = 0; j < n; j++) {  
5      // O(1)  
6  }
```

Le due sezioni non sono annidate ma consecutive, quindi:

$$T(n) = O(n) + O(n) = O(n)$$

Esempio 2 — Cicli annidati e condizione interna:

```
1  for (int i = 0; i < n; i++) {  
2      if (i % 2 == 0) {  
3          for (int j = 0; j < n; j++) {  
4              // O(1)  
5          }  
6      }  
7  }
```

Il ciclo interno si esegue solo per metà delle iterazioni esterne, ma l'ordine di grandezza resta invariato:

$$T(n) = O\left(\frac{n}{2} \times n\right) = O(n^2)$$

Algoritmi con Procedure non ricorsive

La procedura generale che dev'essere seguita per il calcolo della complessità computazionale consiste nei seguenti passi:

- si esamina il programma scendendo all'interno di ciascuna procedura o funzione fino ad individuare quella o quelle che non presentano chiamate ad altre procedure o funzioni al loro interno;
- per le procedure o funzioni che non contengono al loro interno chiamate ad altre procedure o funzioni, devono essere tenute presenti le considerazioni fatte fino ad ora e cioè applicare le regole sui singoli costrutti, in modo da determinare la complessità computazionale delle procedure o funzioni considerate come se fossero semplici blocchi di codice;
- determinata la complessità delle procedure/funzioni al punto precedente si passa a considerare invece la chiamata a queste ultime da parte di altre procedure/funzioni. Per ciascuna di tali procedure e funzioni viene calcolata la complessità computazionale, sostituendo a ciascuna chiamata di procedura o funzione la relativa complessità computazionale;
- si risale all'interno di questa pila di chiamate, ricordando la complessità computazionale calcolata ad ogni livello di risalita. Tale procedimento termina quando è stata calcolata la complessità di tutte le procedure e funzioni presenti nel programma principale;

- la complessità computazionale del *main()* si calcola sostituendo a ciascuna chiamata di procedura o funzione la relativa complessità computazionale, calcolata secondo le regole descritte nei passi precedenti.

Ad esempio, si consideri il seguente programma:

```

1  #include <iostream>
2  using namespace std;
3
4  void stampaVettore(int A[], int n) {
5      for (int i = 0; i < n; i++)
6          cout << A[i] << " ";          // O(1)
7  }
8
9  int sommaVettore(int A[], int n) {
10     int somma = 0;                      // O(1)
11     for (int i = 0; i < n; i++)          // n iterazioni
12         somma += A[i];                  // O(1) per iterazione
13     return somma;                       // O(1)
14 }
15
16 int main() {
17     int n = 100;                        // O(1)
18     int A[100];                        // O(1)
19
20     for (int i = 0; i < n; i++)          // n iterazioni
21         A[i] = i;                      // O(1) per iterazione
22
23     stampaVettore(A, n);                 // O(n)
24     int risultato = sommaVettore(A, n); // O(n)
25
26     cout << "Somma: " << risultato;    // O(1)
27     return 0;
28 }

```

1. La funzione `stampaVettore()` contiene un ciclo `for` che scorre n elementi, quindi:

$$T_{\text{stampaVettore}}(n) = O(n)$$

2. La funzione `sommaVettore()` ha anch'essa un ciclo su n elementi, con operazioni elementari all'interno:

$$T_{\text{sommaVettore}}(n) = O(n)$$

3. La funzione `main()` esegue:

- Un ciclo `for` che inizializza un vettore di n elementi $\rightarrow O(n)$.
- Una chiamata a `stampaVettore()` $\rightarrow O(n)$.
- Una chiamata a `sommaVettore()` $\rightarrow O(n)$.

Applicando la regola della somma:

$$T_{\text{main}}(n) = O(n) + O(n) + O(n) = O(n)$$

Algoritmi con Funzioni Ricorsive

Per calcolare la complessità di un algoritmo ricorsivo non è possibile utilizzare le tecniche che abbiamo già analizzato, ma è necessario introdurre le **relazioni di ricorrenza**; esse descrivono una funzione nei termini del suo valore su input sempre più piccoli.

Ad esempio, consideriamo la funzione classica per calcolare la somma dei primi n numeri:

```
1  #include <iostream>
2  using namespace std;
3
4  // Funzione ricorsiva
5  int somma(int n) {
6      if (n == 0) return 0;      // caso base O(1)
7      return n + somma(n - 1);  // ricorsione
8  }
9
10 int main() {
11     int n = 100;
12     int risultato = somma(n);
13     cout << "Somma: " << risultato;
14     return 0;
15 }
```

Analizzando la complessità, si nota che:

- La funzione `somma()` chiama se stessa una volta per ogni valore di n fino al caso base:

$$T(n) = T(n - 1) + O(1)$$

- Espandendo passo passo:

$$T(n) = O(1) + O(1) + \dots + O(1) \text{ (per } n \text{ volte)} = O(n)$$

- La funzione `main()` esegue:
 - Assegnazione di $n \rightarrow O(1)$
 - Chiamata a `somma(n)` $\rightarrow O(n)$
 - Stampa del risultato $\rightarrow O(1)$

Applicando la regola della somma:

$$T_{\text{main}}(n) = O(1) + O(n) + O(1) = O(n)$$

Consideriamo un altro esempio: la versione ricorsiva della funzione di Fibonacci:

```
1  int fib(int n) {
2      if (n <= 1) return n;
3      return fib(n-1) + fib(n-2);
4  }
```

Possiamo affermare che:

- La funzione chiama se stessa due volte per ogni valore di n fino al caso base:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

- Questa ricorrenza cresce esponenzialmente, poiché ciascuna chiamata produce due nuove chiamate. Di conseguenza:

$$T(n) = O(2^n)$$

1.3 Esercizi di complessità computazionale

1.3.1 Esercizio 1

Considera il seguente codice C++:

```

1  int a = 5, b = 10;
2  int maxVal;
3
4  if (a > b) {
5      maxVal = a;
6      b = b + 1;
7  } else {
8      maxVal = b;
9      a = a + 1;
10     b = b + 2;
11 }
```

Domanda: Calcola la complessità computazionale totale di questo frammento di codice in termini di O .

SOLUZIONE

Analizziamo il frammento di codice riga per riga:

- `int a = 5, b = 10;` → $O(1)$ (assegnazione di variabili)
- `int maxVal;` → $O(1)$ (assegnazione di variabile)
- `if (a > b)` → $O(1)$ (confronto)

Ramo vero:

- `maxVal = a;` → $O(1)$
- `b = b + 1;` → $O(1)$

Ramo falso:

- `maxVal = b;` → $O(1)$
- `a = a + 1;` → $O(1)$
- `b = b + 2;` → $O(1)$

Somma complessiva: Ogni ramo contiene un numero costante di operazioni $O(1)$. Applicando la regola della somma:

$$T(n) = O(1) + O(1) + \max(O(1) + O(1), O(1) + O(1) + O(1)) = O(1)$$

Conclusione: Il frammento di codice ha complessità totale $O(1)$.

1.3.2 Esercizio 2

Considera il seguente frammento di codice in C++:

```
1  int sum = 0;
2  for (int i = 0; i < n; i++) {
3      if (i % 2 == 0) {
4          sum += i;
5      } else {
6          sum -= i;
7      }
8  }
```

Domanda: Determinare la complessità computazionale del frammento di codice sopra in termini di n . Considerando un dispositivo capace di fare 10^8 operazioni al secondo e considerando un input $n < 10^4$, il tempo di esecuzione dell'algoritmo è inferiore a 1 secondo?

SOLUZIONE

Analizziamo ancora una volta il frammento di codice riga per riga:

- `int sum = 0` $\rightarrow O(1)$
- Il ciclo `for(int i = 0; i < n; i++)` viene eseguito n volte:
quindi il ciclo contribuisce con $n \cdot T_{\text{interno}}$.
- All'interno del ciclo, abbiamo un `if-else`:
 - Condizione `i % 2 == 0` $\rightarrow O(1)$
 - Operazioni nel ramo vero `sum += i` $\rightarrow O(1)$
 - Operazioni nel ramo falso `sum -= i` $\rightarrow O(1)$

Quindi ogni iterazione del ciclo ha complessità:

$$T_{\text{iterazione}} = O(1) + O(1) = O(1)$$

- Moltiplicando per il numero di iterazioni:

$$T_{\text{ciclo}} = n \cdot O(1) = O(n)$$

- Sommando tutte le parti del codice:

$$T_{\text{totale}} = O(1) + O(n) = O(n)$$

Conclusion: La complessità complessiva del frammento di codice è $O(n)$.

$$t_{\text{esecuzione}} \approx \frac{n}{10^8} < \frac{10^4}{10^8} = 10^{-4} \text{ secondi}$$

Poiché $10^{-4} \ll 1$, possiamo concludere che l'algoritmo termina molto prima di 1 secondo.

1.3.3 Esercizio 3

Consideriamo il seguente codice C++:

```
1  int sum = 0;
2  for (int i = 0; i < n; i++) {
3      for (int j = 0; j < m; j++) {
4          sum += i + j;
5      }
6  }
```

Domanda: Determinare la complessità computazionale del frammento di codice sopra in termini di n . Considerando un dispositivo capace di fare 10^8 operazioni al secondo e considerando un input $n < 10^4$ e $m < 10^3$, il tempo di esecuzione dell'algoritmo è inferiore a 1 secondo?

Analisi della complessità:

- $\text{int sum} = 0 \rightarrow O(1)$
- Il ciclo esterno `for(int i = 0; i < n; i++)` viene eseguito n volte: quindi il ciclo contribuisce con $n \cdot T_{\text{interno}}$.
- Il ciclo interno `for(int j = 0; j < m; j++)` viene eseguito m volte: quindi la complessità è $m \cdot T_{\text{interno}}$
- Nel ciclo interno abbiamo un'operazione elementare: `sum += i+j` $\rightarrow O(1)$.
- Applicando la regola del prodotto per i cicli annidati:

$$T_{\text{ciclo}}(n, m) = n \cdot (m \cdot O(1)) = O(n \cdot m)$$

- Sommando l'assegnazione iniziale:

$$T_{\text{totale}}(n, m) = O(1) + O(n \cdot m) = O(n \cdot m)$$

Conclusione: La complessità complessiva del frammento di codice è

$$T(n, m) = O(n \cdot m)$$

Il tempo di esecuzione è:

$$t_{\text{esecuzione}} \approx \frac{n \cdot m}{10^8} < \frac{10^4 \cdot 10^3}{10^8} = \frac{10^7}{10^8} = 0.1 \text{ secondo}$$

Da cui ne deriva che il tempo di esecuzione è meno di 1 secondo (ma non di molto).

Capitolo 2

Strutture dati in C++

2.1 Vector

Il **vector** è una struttura dati della Standard Template Library (STL) che rappresenta un **array dinamico**. A differenza degli array statici, la sua dimensione può crescere o ridursi durante l'esecuzione del programma. È contenitore sequenziale, quindi mantiene l'ordine degli elementi inseriti.

Dichiarazione di un vector:

```
1  #include <vector>
2  #include <iostream>
3  using namespace std;
4
5  vector<int> v;           // vector vuoto di interi
6
7  vector<int> v2(10);      // vector di 10 elementi
                           //   inizializzati a 0
8
9  vector<int> v3(5, 7);    // vector di 5 elementi inizializzati
                           //   a 7
```

Prima di andare ai metodi di questa struttura, è opportuno spendere due parole sugli iteratori.

2.1.1 Iterazione tramite `begin()` e `end()`

I metodi `begin()` ed `end()` sono due funzioni fondamentali dei contenitori della STL, come **vector**, e servono per ottenere **iteratori** che consentono di scorrere gli elementi del contenitore.

- `v.begin()` Restituisce un iteratore che punta al **primo elemento** del **vector**. È come un "puntatore" all'inizio della sequenza di dati.
- `v.end()` Restituisce un iteratore che punta **alla posizione successiva all'ultimo elemento** del **vector**. Non rappresenta quindi un elemento valido, ma serve per **indicare la fine** del contenitore.

L'uso tipico di `begin()` e `end()` è all'interno di un ciclo, per visitare tutti gli elementi del **vector**:

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<int> v = {10, 20, 30, 40};
7
8      for (auto it = v.begin(); it != v.end(); ++it) {
9          cout << *it << " ";
10     }
11     return 0;
12 }

```

In questo esempio:

- `auto it = v.begin();` inizializza l'iteratore al primo elemento^{1 2};
- il ciclo prosegue finché `it` non raggiunge `v.end()`, cioè finché ci sono elementi validi;
- `*it` consente di accedere al valore dell'elemento puntato.

Nota: `v.end()` non punta a un elemento valido, quindi non bisogna mai dereferenziarlo (cioè non fare `*v.end()`), poiché ciò porterebbe a un comportamento indefinito.

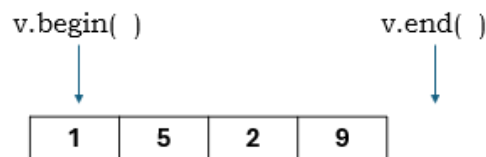


Figura 2.1: Iteratori `v.begin()` e `v.end()`

2.1.2 Operazioni principali sui vector

- **Accesso agli elementi:** `v[i]`

```

1  int x = v[1];           // accesso diretto           (senza
    cout << x;             // stampa l'elemento di v all'   controllo)
2  1                         // indice

```

Complessità: $O(1)$.

- **Aggiunta in coda:** `push_back(x)`

```

1  v.push_back(3); // aggiunge 3 alla fine

```

Complessità: $O(1)$.

¹È stato usato un **auto it** in quanto `it` viene confrontato con l'iteratore `v.end()` ed è opportuno che questi siano dello stesso tipo.

²In alternativa, avrei potuto usare `vector<int>::iterator it`

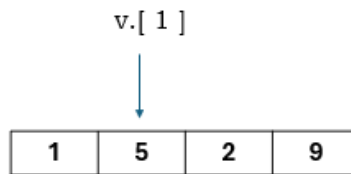


Figura 2.2: Accesso ad un elemento del vettore

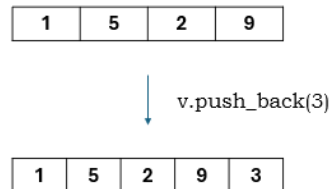


Figura 2.3: Inserimento di un elemento alla fine del vector

- **Rimozione dall'ultima posizione: `pop_back()`**

```
1 v.pop_back(); // rimuove l'ultimo elemento
```

Complessità: $O(1)$.

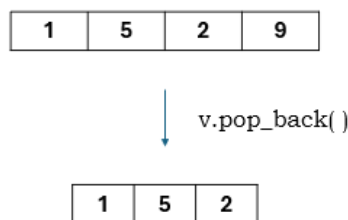


Figura 2.4: Cancellazione dell'ultimo elemento del vettore

- **Dimensione: `v.size()`** Restituisce il numero di elementi presenti nel vector.

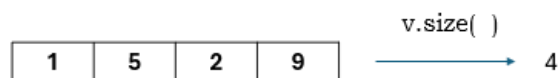


Figura 2.5: Numero di elementi nel vettore

- **Controllo se vuoto: `v.empty()`** Restituisce `true` se il vector non contiene elementi.

2.1.3 Inserimento ed eliminazione in posizioni specifiche

```
1 v.insert(v.begin() + 2, 10); // inserisce 10 in posizione 2 e
                              // shifta a destra tutti gli altri elementi
2
3 v.erase(v.begin() + 2);    // elimina elemento in posizione 2 e
                              // shifta a sinistra tutti gli altri
                              // elementi
```

Note:

- Inserimenti o cancellazioni in mezzo al vector hanno complessità $O(n)$, perché gli elementi successivi devono essere spostati.

2.1.4 Altre operazioni utili

- `v.clear()` → svuota il vector, $O(n)$
- `v.front()` → accede al primo elemento, $O(1)$
- `v.back()` → accede all'ultimo elemento, $O(1)$
- `v.resize(k)` → cambia la dimensione a k elementi (elimina quelli di troppo)

2.1.5 Iterazione su un vector

```

1 // Utilizzo di un ciclo for tradizionale
2 for (size_t i = 0; i < v.size(); i++) {
3     cout << v[i] << " ";
4 }
5
6 // Utilizzo del range-based for (C++11+)
7 for (int x : v) {
8     cout << x << " ";
9 }
10
11 // Utilizzo di iteratori
12 for (vector<int>::iterator it = v.begin(); it != v.end(); ++it) {
13     cout << *it << " ";
14 }

```

Note:

- la variabile di controllo del `for` è stata dichiarata come tipo `size_t`. Questo perché verrà confrontata con `v.size()` che, di fatto, è anch'essa di tipo `size_t`.
- `int x : v` è una sintassi valida da C++11 in poi. Molto semplicemente, `x` assume il valore di ogni elemento del vector `v`. In altre parole `x` è una copia e, come tale, non fa cambiare il valore degli elementi di `v` in caso scrivessi `x = 2`.

2.1.6 Complessità computazionale riassunta

Operazione	Complessità
Accesso a un elemento <code>v[i]</code>	$O(1)$
Inserimento in mezzo <code>v.insert(iteratore, valore)</code>	$O(n)$
Cancellazione in mezzo <code>v.erase(iteratore)</code>	$O(n)$
Aggiunta in coda <code>v.push_back(valore)</code>	$O(1)$
Eliminazione ultimo elemento <code>v.pop_back()</code>	$O(1)$
Dimensione <code>v.size()</code>	$O(1)$

Tabella 2.1: Complessità operazioni principali sui vector

2.2 Map

La **map** è una struttura dati della Standard Template Library (STL) che rappresenta un **contenitore associativo ordinato**. Permette di memorizzare coppie chiave-valore e garantisce che le chiavi siano **uniche** e **automaticamente ordinate**. Per accedere al valore, quindi, ho bisogno della chiave.

In figura 2.6 si può notare come le chiavi (`int`) siano ordinate in ordine crescente e siano associate a dei valori (`string`)

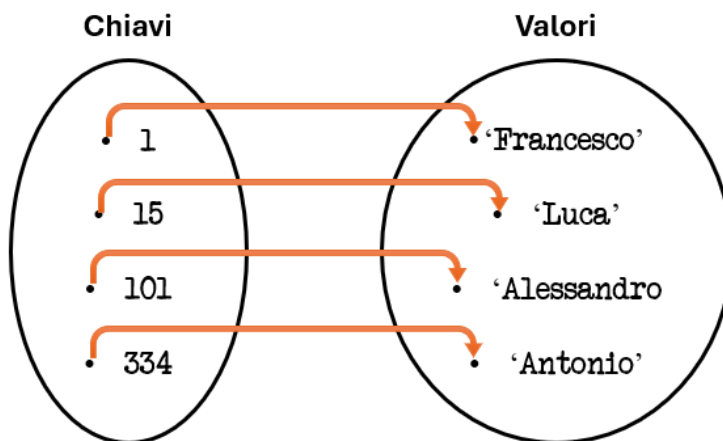


Figura 2.6: Rappresentazione grafica di una **map**

2.2.1 Dichiarazione di una map

```
1  #include <map>
2  #include <iostream>
3  using namespace std;
4
5  map<string, int> m1;           // mappa con chiavi stringa e valori
    interi
6
7  map<int, string> m2 = {       // mappa inizializzata con le coppie
                                // chiave-valore (le coppie saranno
                                // ordinate in ordine di chiave
                                // crescente automaticamente)
8      {101, "Alessandro"},
9      {334, "Antonio"},
10     {1, "Francesco"},
11     {15, "Luca"}
12 };
13
14 map<int, string, greater<int>> m3 = {
15     {1, "uno"},
16     {3, "tre"},
17     {2, "due"}
18 }; // greater<int> indica alla mappa di ordinare le chiavi dalla
    piu' grande al piu' piccola
```

Note:

- Da notare che nella riga 7 del codice, è stata inizializzata una mappa inserendo delle coppie chiave-valore non ordinate.
Sarà la struttura stessa ad ordinarle in base alle chiavi, in ordine crescente.
- Nella riga 14, invece, è stato fatto uso di **greater<int>** che ordina la mappa per chiavi decrescenti (se la chiave fosse stata **string** avrei dovuto usare **greater<string>**)

In un certo senso, si può vedere una map come un array associativo dinamico: invece di accedere agli elementi tramite un indice numerico, si accede tramite una chiave qualsiasi (int o string, ecc)

2.2.2 Operazioni principali sulle map

Le operazioni che seguono fanno riferimento alla mappa della figura 2.6.

- **Accesso ad un elemento** tramite chiave

```
1      cout << m[1];           // stampa "Francesco"
```

Complessità: $O(\log n)$

Nota: se la chiave non esiste, l'operatore `[]` crea automaticamente un nuovo elemento con valore di default.

- **Inserimento di un elemento:** `m.insert` o assegnamento tramite chiave

```
1      m[3] = "Luigi";           // inserisce la coppia  
                                     (3, "Luigi")  
2      m.insert({2, "Anna"});    // inserisce la coppia (2,  
                                     "Anna")
```

Complessità: $O(\log n)$ per ciascun inserimento. Bisogna infatti ricordare che le mappe sono ordinate e, quindi l'inserimento va messo in posizione corretta.

Nota: l'`insert` viene ignorato se si prova ad inserire un valore attraverso una chiave già esistente.

- **Rimozione di un elemento:** `erase`

```
1      m.erase(1);              // rimuove la chiave 1 e  
                                     il relativo valore
```

Complessità: $O(\log n)$

- **Dimensione:** `m.size()` Restituisce il numero di coppie chiave-valore nella mappa. Nel caso della figura 2.6:

```
1      cout << m.size();         // stampa 4
```

Complessità: $O(1)$

- **Cercare un elemento:** `m.find(key)`

Questo metodo cerca la chiave **key** nella mappa. Se la trova restituisce un iteratore alla coppia, altrimenti restituisce `m.end()`.

```

1      #include <iostream>
2      #include <map>
3      using namespace std;
4
5      int main() {
6          map<int, string> m = {
7              {1, "uno"},
8              {2, "due"},
9              {3, "tre"}
10         };
11
12         int chiave = 2;
13
14         // m.find(chiave) restituisce un iteratore alla coppia
            (chiave, valore) se la chiave esiste, altrimenti
            restituisce m.end().
15         auto it = m.find(chiave);
16
17         if (it != m.end()) {
18             // (*it).first      la chiave, (*it).second      il
            valore
19             cout << "Trovato: " << it->first << " -> " << it->
            second << endl;
20         } else {
21             cout << "Chiave non trovata." << endl;
22         }
23
24         return 0;
25     }

```

Complessità: $O(\log n)$

Nota: `(*it.first` e `it->first`) sono la stessa cosa.

- **Controllo se vuota:** `m.empty()` Restituisce `true` se la map non contiene elementi.

2.2.3 Altre operazioni utili

- `m.count(key)` → restituisce 1 se la chiave esiste, 0 altrimenti.

2.2.4 Iterazione su una map

```

1      for (auto it = m.begin(); it != m.end(); ++it) {
2          cout << it->first << " -> " << it->second << endl;
3      }
4
5      // range-based for (C++11+)
6      for (auto [key, value] : m) {
7          cout << key << " -> " << value << endl;
8      }

```


Note:

- `it->first` accede alla chiave, `it->second` al valore.
- Gli elementi sono ordinati automaticamente in base alla chiave (in ordine crescente di default, cioè secondo l'operatore `<` della chiave).

2.2.5 Complessità computazionale riassunta

Operazione	Complessità
Accesso <code>m[i]</code>	$O(\log n)$
Ricerca <code>m.find(key)</code>	$O(\log n)$
Inserimento <code>m.insert(key, value)</code>	$O(\log n)$
Cancellazione <code>m.erase(key)</code>	$O(\log n)$
Dimensione <code>m.size()</code>	$O(1)$
Controllo se vuota <code>m.empty()</code>	$O(1)$

Tabella 2.2: Complessità operazioni principali sulle map

2.3 Set

Il **set** è una struttura dati della Standard Template Library (STL) che rappresenta un **contenitore associativo ordinato** di elementi **unici**. Ogni elemento è sia la chiave che il valore, e viene automaticamente ordinato in base al suo valore.

In figura 2.7 si può notare come gli elementi siano ordinati in modo crescente e non siano presenti duplicati.

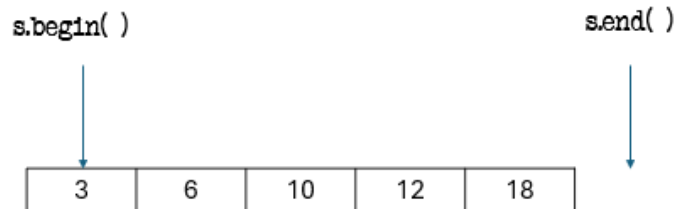


Figura 2.7: Rappresentazione grafica di un **set**

2.3.1 Dichiarazione di un set

```
1  #include <set>
2  #include <iostream>
3  using namespace std;
4
5  set<int> s1;           // set vuoto di interi
6
7  set<int> s2 = {5, 1, 9, 1, 3}; // inizializzazione con duplicati
8  // Risultato: {1, 3, 5, 9} (gli elementi duplicati vengono
9  // ignorati)
10
11 set<string, greater<string>> s3 = {"mela", "pera", "banana"};
// greater<string> ordina in modo decrescente
```

Note:

- Gli elementi di un set sono sempre univoci.
- La struttura mantiene automaticamente l'ordine.
- Se si vuole un set non ordinato, si può usare `unordered_set`.

2.3.2 Operazioni principali sui set

Le operazioni che seguono fanno riferimento al set della figura 2.7.

- **Inserimento di un elemento:** `s.insert(valore)`

```
1  s.insert(4);           // inserisce l'elemento 4
2  s.insert(3);           // ignorato se 3 e' gia' presente
```

Complessità: $O(\log n)$

- **Rimozione di un elemento:** `s.erase(valore)`

```
1      s.erase(1);          // rimuove l'elemento 1 se presente
```

Complessità: $O(\log n)$

- **Ricerca di un elemento:** `s.find(valore)`

```
1      auto it = s.find(3);
2      if (it != s.end())
3          cout << "Trovato: " << *it;
4      else
5          cout << "Elemento non trovato";
```

Complessità: $O(\log n)$

- **Verifica esistenza di un elemento:** `s.count(valore)`

```
1      if (s.count(5)) cout << "Presente!";
2      else cout << "Assente!";
```

Restituisce 1 se l'elemento esiste, 0 altrimenti. Complessità: $O(\log n)$

- **Dimensione:** `s.size()`

```
1      cout << s.size();    // numero di elementi nel set
```

Complessità: $O(1)$

- **Controllo se vuoto:** `s.empty()`

```
1      if (s.empty()) cout << "Set vuoto";
```

Complessità: $O(1)$

Accesso ad un elemento:

L'accesso ad un elemento di un **set** avviene mediante dereferenziazione dell'iteratore.

Questa volta però, a differenza dei **vector**, non possiamo accedere ad un elemento sommando un intero all'iteratore:

```
1      #include <iostream>
2      #include <set>
3      using namespace std;
4
5      int main() {
6          set<int> s = {1, 3, 5, 7};
7
8          it = s.begin();
9
10         // voglio stampare il 4 elemento
11         cout << s.begin() + 3; // stampa un errore!
12
13         return 0;
14     }
```

L'errore è dovuto al fatto che gli iteratori dei **set** non supportano l'aritmetica dei puntatori. Possiamo solo avanzare o retrocedere di una posizione alla volta (con `it++` e `it--`).

Quindi se volessi accedere al terzo elemento, una soluzione potrebbe essere:

```

1      #include <iostream>
2      #include <set>
3      using namespace std;
4
5      int main() {
6          set<int> s = {1, 3, 5, 7};
7
8          it = s.begin();
9
10         it++;
11         it++;
12
13         // Attenzione: fare it = it + 2      errato in quanto gli
            iteratori dei set non supportano l'aritmetica dei
            puntatori!
14
15         cout << *it; // Stampa 5
16
17         return 0;
18     }

```

Un'alternativa è utilizzare la libreria `<iterator>`, che mette a disposizione la funzione **advance(iterator, n)**. Questa funzione permette di spostare l'iteratore di `n` posizioni lungo il contenitore.

È doveroso notare che la funzione `advance(valore, n)` può andare ben oltre la posizione `s.end()`. Ciò significa che è opportuno verificare che il set abbia elementi sufficienti per accedere alla posizione desiderata:

```

1      #include <iostream>
2      #include <set>
3      #include <iterator>
4      using namespace std;
5
6      int main() {
7          set<int> s = {1, 3, 5, 7, 9};
8
9          // controlliamo prima che il set abbia almeno 3
            elementi
10         if (s.size() >= 3) {
11             auto it = s.begin();          // iteratore all'inizio
12             advance(it, 2);                // spostiamo l'iteratore
            di 2 posizioni (3 elemento)
13             cout << "Il terzo elemento e': " << *it << endl;
14         } else {
15             cout << "Il set ha meno di 3 elementi." << endl;
16         }
17
18         return 0;
19     }

```

2.3.3 Altre operazioni utili

- **lower_bound**

Restituisce un iteratore al primo elemento **maggiore o uguale** al valore passato come parametro. Restituisce `s.end()` se non trova nulla.

```
1      #include <iostream>
2      #include <set>
3      using namespace std;
4
5      int main() {
6          set<int> s = {1, 3, 5, 7};
7
8          auto it = s.lower_bound(3); // primo elemento >= 3
9          if(it != s.end())
10             cout << "lower_bound(3) = " << *it << endl;
11
12             return 0;
13     }
```

Output: lower_bound(3) = 3

- **upper_bound**

Restituisce un iteratore al primo elemento **maggiore** del valore passato come parametro. `s.end()` se non trova nulla.

```
1      #include <iostream>
2      #include <set>
3      using namespace std;
4
5      int main() {
6          set<int> s = {1, 3, 5, 7};
7
8          auto it = s.upper_bound(3); // primo elemento > 3
9          if(it != s.end())
10             cout << "upper_bound(3) = " << *it << endl;
11
12             return 0;
13     }
```

Output: upper_bound(3) = 5

2.3.4 Iterazione su un set

```
1   for (auto it = s.begin(); it != s.end(); ++it)
2       cout << *it << " ";
3
4   // range-based for (C++11+)
5   for (auto x : s)
6       cout << x << " ";
```

Note:

- L'iteratore restituisce il valore direttamente, non una coppia chiave-valore.
- Gli elementi sono ordinati automaticamente.

2.3.5 Esempio pratico

```
1   #include <set>
2   #include <iostream>
3   using namespace std;
4
5   int main() {
6       set<int> s;
7
8       s.insert(10);
9       s.insert(5);
10      s.insert(10);
11      s.insert(8);
12
13      cout << "Elementi nel set: ";
14      for (int x : s) cout << x << " ";
15
16      cout << "\nDimensione: " << s.size() << endl;
17
18      if (s.find(5) != s.end())
19          cout << "5 e' presente!\n";
20
21      s.erase(8);
22      cout << "Dopo cancellazione: ";
23      for (int x : s) cout << x << " ";
24
25      return 0;
26  }
```

Output:

```
Elementi nel set: 5 8 10
Dimensione: 3
5 e' presente!
Dopo cancellazione: 5 10
```

2.3.6 Complessità computazionale riassunta

In tabella 2.3 sono state riassunte le complessità computazionali delle operazioni sui **set**³

Operazione	Complessità
Inserimento <code>s.insert(x)</code>	$O(\log n)$
Ricerca <code>s.find(x)</code>	$O(\log n)$
Cancellazione <code>s.erase(x)</code>	$O(\log n)$
Controllo esistenza <code>s.count(x)</code>	$O(\log n)$
<code>lower_bound</code> <code>s.lower_bound(x)</code>	$O(\log n)$
<code>upper_bound</code> <code>s.upper_bound(x)</code>	$O(\log n)$
Dimensione <code>s.size()</code>	$O(1)$
Controllo se vuoto <code>s.empty()</code>	$O(1)$

Tabella 2.3: Complessità operazioni principali sui set

³La funzione `advance(it, n)` ha complessità $O(n)$

2.4 Multiset

Il **multiset** è una struttura dati della libreria standard di C++ molto simile al **set**, con una differenza fondamentale: mentre nel **set** ogni elemento è unico, nel **multiset** possono esistere più copie dello stesso valore.

```
1  #include <iostream>
2  #include <set>
3  using namespace std;
4
5  int main() {
6      multiset<int> ms;
7      ms.insert(5);
8      ms.insert(5);
9      ms.insert(3);
10     ms.insert(7);
11
12     for (int x : ms)
13         cout << x << " "; // stampa: 3 5 5 7
14 }
```

Come il **set**, anche il **multiset** memorizza gli elementi in modo ordinato e permette di eseguire operazioni di inserimento, ricerca e cancellazione in $O(\log n)$.

2.4.1 Caratteristiche principali

- Gli elementi sono **ordinati automaticamente**.
- È possibile inserire **valori duplicati**.
- Non è possibile accedere agli elementi tramite indice.
- Gli iteratori sono bidirezionali, quindi non supportano operazioni aritmetiche ($it + n$).

2.4.2 Esempio pratico

```
1  #include <iostream>
2  #include <set>
3  using namespace std;
4
5  int main() {
6      multiset<int> ms = {2, 3, 3, 5, 5, 5};
7
8      cout << "Numero di 5: " << ms.count(5) << endl; // 3
9
10     ms.erase(ms.find(3)); // cancella solo una delle due occorrenze
11                             di 3
12
13     for (int x : ms)
14         cout << x << " "; // stampa: 2 3 5 5 5
15 }
```


2.4.3 Complessità computazionale riassunta

Operazione	Complessità
Inserimento <code>ms.insert(x)</code>	$O(\log n)$
Ricerca <code>ms.find(x)</code>	$O(\log n)$
Cancellazione di un elemento <code>ms.erase(it)</code>	$O(1)$
Cancellazione di tutti gli elementi con un certo valore <code>ms.erase(x)</code>	$O(\log n + k)^4$
Conteggio occorrenze <code>ms.count(x)</code>	$O(\log n + k)$
<code>lower_bound</code> <code>ms.lower_bound(x)</code>	$O(\log n)$
<code>upper_bound</code> <code>ms.upper_bound(x)</code>	$O(\log n)$
Dimensione <code>ms.size()</code>	$O(1)$
Controllo se vuoto <code>ms.empty()</code>	$O(1)$

Tabella 2.4: Complessità operazioni principali sui multiset

⁴Nella tabella delle complessità del multiset, la lettera **k** indica il numero di elementi con lo stesso valore (cioè il numero di duplicati). Infatti `ms.erase(x)` deve cancellare **k** occorrenze e, di conseguenza, la complessità risulta $O(\log n)$ per la ricerca più **k** per eliminare ogni occorrenza)

2.5 Stack

Lo **stack** (in italiano: pila) è una struttura dati della Standard Template Library (STL) che segue il principio **LIFO** (*Last In, First Out*), ovvero l'ultimo elemento inserito è il primo a essere rimosso.

In figura 2.8 è mostrata la logica di funzionamento di una pila: gli elementi vengono aggiunti (push) e rimossi (pop) sempre dalla stessa estremità.

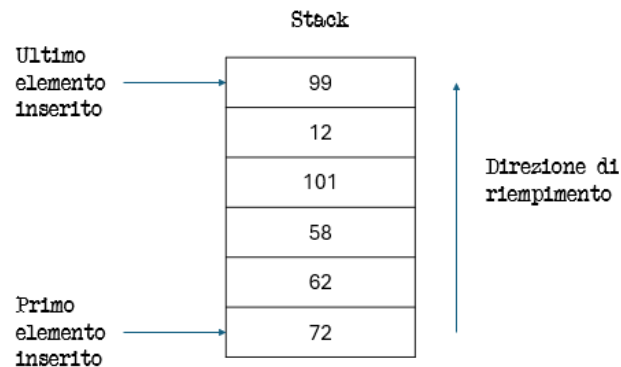


Figura 2.8: Rappresentazione grafica di uno **stack**

2.5.1 Dichiarazione di uno stack

```
1  #include <stack>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      stack<int> s;      // dichiara uno stack di interi vuoto
7
8      s.push(10);        // inserisce 10
9      s.push(20);        // inserisce 20
10     s.push(30);        // inserisce 30
11
12     cout << "Top: " << s.top() << endl; // stampa 30
13
14     s.pop();            // rimuove l'ultimo elemento inserito (30)
15
16     cout << "Top dopo pop: " << s.top() << endl; // stampa 20
17 }
```

Note:

- Lo stack non consente accesso diretto agli elementi (come con gli indici).
- È possibile accedere solo all'elemento in cima (`top()`).
- La classe **stack** è un *adattatore di contenitore*, che di default usa un **deque** come contenitore sottostante.

2.5.2 Operazioni principali sullo stack

Le operazioni principali disponibili sono:

- **push(valore)** – Inserisce un nuovo elemento in cima alla pila.

```
1 s.push(42);
```

Complessità: $O(1)$

- **pop()** – Rimuove l'elemento in cima alla pila.

```
1 s.pop();
```

Complessità: $O(1)$

- **top()** – Restituisce una **riferimento** all'elemento in cima.

```
1 cout << s.top();
```

Complessità: $O(1)$

- **empty()** – Verifica se la pila è vuota.

```
1 if (s.empty()) cout << "Stack vuoto!";
```

Complessità: $O(1)$

- **size()** – Restituisce il numero di elementi presenti nello stack.

```
1 cout << s.size();
```

Complessità: $O(1)$

2.5.3 Iterazione su uno stack

Lo **stack non supporta l'iterazione diretta** (come con `for(auto x : s)` o iteratori), poiché è pensato per un accesso controllato di tipo LIFO. Tuttavia, è possibile svuotarlo progressivamente per leggere tutti i suoi elementi:

```
1 #include <stack>
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     stack<int> s;
7     s.push(10);
8     s.push(20);
9     s.push(30);
10
11     while (!s.empty()) {
12         cout << s.top() << " ";
13         s.pop();
14     }
15     // Output: 30 20 10
16 }
```

Nota: Questo metodo svuota lo stack. Se serve mantenerlo, conviene copiarlo prima in un altro stack.

2.5.4 Esempio pratico

```
1  #include <stack>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      stack<string> history;
7
8      history.push("www.google.com");
9      history.push("www.openai.com");
10     history.push("www.github.com");
11
12     cout << "Pagina corrente: " << history.top() << endl;
13     history.pop();
14
15     cout << "Torno a: " << history.top() << endl;
16
17     cout << "Totale pagine: " << history.size() << endl;
18 }
```

Output:

Pagina corrente: www.github.com
Torno a: www.openai.com
Totale pagine: 2

2.5.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento <code>s.push(x)</code>	$O(1)$
Rimozione <code>s.pop()</code>	$O(1)$
Accesso al top <code>s.top()</code>	$O(1)$
Controllo se vuoto <code>s.empty()</code>	$O(1)$
Dimensione <code>s.size()</code>	$O(1)$

Tabella 2.5: Complessità operazioni principali sullo stack

2.6 Queue

La **queue** (in italiano: coda) è una struttura dati della STL che segue il principio **FIFO** (*First In, First Out*), ovvero il primo elemento inserito è il primo a essere rimosso. Si comporta come una vera e propria coda di persone: chi entra per primo esce per primo.

Di fatto è il contrario di uno **stack**

In figura 2.9 è illustrato il funzionamento logico della coda.

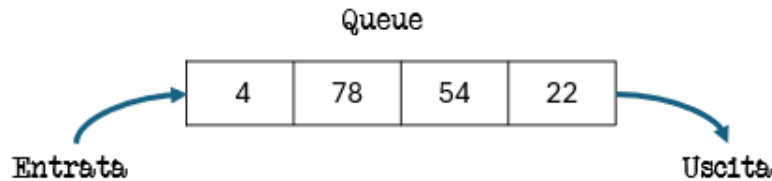


Figura 2.9: Rappresentazione grafica di una **queue**

2.6.1 Dichiarazione di una queue

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      queue<int> q;          // dichiara una coda di interi
7
8      q.push(10);            // inserisce 10 in fondo alla coda
9      q.push(20);            // inserisce 20
10     q.push(30);            // inserisce 30
11
12     cout << "Front: " << q.front() << endl; // stampa 10
13     cout << "Back: " << q.back() << endl;   // stampa 30
14
15     q.pop();                // rimuove il primo elemento (10)
16
17     cout << "Front dopo pop: " << q.front() << endl; // stampa 20
18 }
```

Note:

- È possibile accedere solo all'elemento **front** (testa) e a quello **back** (coda).
- Non è possibile accedere direttamente agli elementi centrali o iterare sulla coda.
- Internamente, la **queue** è un **adattatore di contenitore**, che usa di default un deque.

2.6.2 Operazioni principali sulla queue

- **push(valore)** – Inserisce un elemento in fondo alla coda.

```
1  q.push(42);
```

Complessità: $O(1)$

- **pop()** – Rimuove l'elemento in testa alla coda.

```
1 q.pop();
```

Complessità: $O(1)$

- **front()** – Restituisce una **riferimento** all'elemento in testa.

```
1 cout << q.front();
```

Complessità: $O(1)$

- **back()** – Restituisce una **riferimento** all'ultimo elemento inserito.

```
1 cout << q.back();
```

Complessità: $O(1)$

- **empty()** – Verifica se la coda è vuota.

```
1 if (q.empty()) cout << "Coda vuota!";
```

Complessità: $O(1)$

- **size()** – Restituisce il numero di elementi nella coda.

```
1 cout << q.size();
```

Complessità: $O(1)$

2.6.3 Iterazione su una queue

Come lo **stack**, anche la **queue** non supporta iteratori né accesso diretto. È possibile tuttavia svuotarla per leggere i valori in ordine di inserimento:

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      queue<int> q;
7      q.push(10);
8      q.push(20);
9      q.push(30);
10
11     while (!q.empty()) {
12         cout << q.front() << " ";
13         q.pop();
14     }
15     // Output: 10 20 30
16 }
```

Nota: Questo metodo svuota la coda. Se serve mantenere i dati, conviene copiarla prima in un'altra queue.

2.6.4 Esempio pratico

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      queue<string> clienti;
7
8      clienti.push("Marco");
9      clienti.push("Giulia");
10     clienti.push("Antonio");
11
12     cout << "Serve: " << clienti.front() << endl;
13     clienti.pop();
14
15     cout << "Prossimo cliente: " << clienti.front() << endl;
16     cout << "Ultimo in coda: " << clienti.back() << endl;
17 }
```

Output:

Serve: Marco
Prossimo cliente: Giulia
Ultimo in coda: Antonio

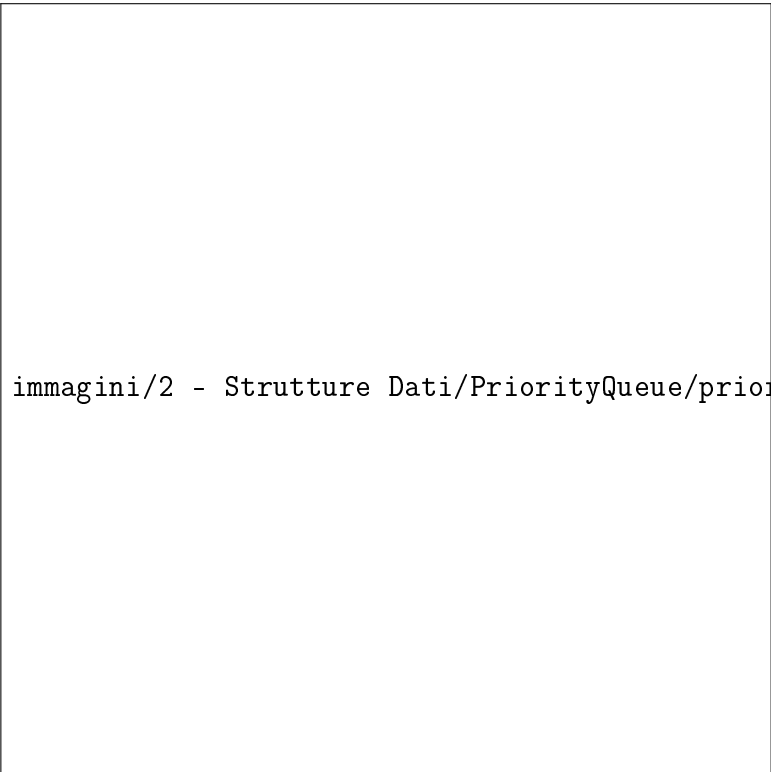
2.6.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento in coda <code>q.push(x)</code>	$O(1)$
Rimozione in testa <code>q.pop()</code>	$O(1)$
Accesso al primo elemento <code>q.front()</code>	$O(1)$
Accesso all'ultimo elemento <code>q.back()</code>	$O(1)$
Controllo se vuota <code>q.empty()</code>	$O(1)$
Dimensione <code>q.size()</code>	$O(1)$

Tabella 2.6: Complessità operazioni principali sulla queue

2.7 Priority Queue

La **priority_queue** è una struttura dati della STL che memorizza gli elementi secondo una **priorità**. A differenza della **queue** tradizionale, non segue l'ordine FIFO, ma estrae sempre l'elemento con la priorità più alta (di default il valore massimo).



immagini/2 - Strutture Dati/PriorityQueue/priority_queue.png

Figura 2.10: Rappresentazione grafica di una **priority_queue**

2.7.1 Dichiarazione di una **priority_queue**

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      priority_queue<int> pq; // priority_queue di interi (max-heap)
7
8      pq.push(10);
9      pq.push(5);
10     pq.push(20);
11
12     cout << "Elemento con priorit massima: " << pq.top() << endl;
13     // 20
14
15     priority_queue<string> pq2;
16
17     pq2.push("banana");
18     pq2.push("mela");
19     pq2.push("arancia");
```



```

20     cout << "Massimo: " << pq2.top() << endl;
21     // stampa mela in quanto mela      considerata massima.
        Confrontando le stringhe alfabeticamente, viene dopo banana
        e arancia.
22
23 }

```

Note:

- Gli elementi sono ordinati automaticamente in base alla priorità.
- Di default è una **max-heap** (massimo elemento in cima).
- Per creare una **min-heap** si usa:

```

1     priority_queue<int, vector<int>, greater<int>> pq_min;

```

2.7.2 Operazioni principali sulla priority_queue

- **push(valore)** – Inserisce un elemento nella coda di priorità.

```

1     pq.push(42);

```

Complessità: $O(\log n)$

- **pop()** – Rimuove l'elemento con priorità più alta.

```

1     pq.pop();

```

Complessità: $O(\log n)$

- **top()** – Restituisce un riferimento all'elemento con priorità più alta.

```

1     cout << pq.top();

```

Complessità: $O(1)$

- **empty()** – Verifica se la coda è vuota.

```

1     if (pq.empty()) cout << "Priority queue vuota!";

```

Complessità: $O(1)$

- **size()** – Restituisce il numero di elementi presenti.

```

1     cout << pq.size();

```

Complessità: $O(1)$

2.7.3 Iterazione su una priority_queue

Così come la queue, non è possibile iterare direttamente o accedere agli elementi centrali.

2.7.4 Esempio pratico

```
1  #include <queue>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      priority_queue<int> pq;
7
8      pq.push(15);
9      pq.push(30);
10     pq.push(10);
11
12     cout << "Massimo elemento: " << pq.top() << endl; // 30
13     pq.pop();
14
15     cout << "Nuovo massimo: " << pq.top() << endl; // 15
16     cout << "Dimensione: " << pq.size() << endl; // 2
17 }
```

Output:

```
Massimo elemento: 30
Nuovo massimo: 15
Dimensione: 2
```

2.7.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento <code>pq.push(x)</code>	$O(\log n)$
Rimozione del massimo <code>pq.pop()</code>	$O(\log n)$
Accesso al massimo <code>pq.top()</code>	$O(1)$
Controllo se vuota <code>pq.empty()</code>	$O(1)$
Dimensione <code>pq.size()</code>	$O(1)$

Tabella 2.7: Complessità operazioni principali sulla `priority_queue`

Nota: la `priority_queue` è molto efficiente quando in un insieme di dati dobbiamo accedere frequentemente all'elemento massimo. Se invece è necessario poter iterare sulla struttura o accedere ad elementi “centrali”, è più conveniente utilizzare un `multiset`.

2.8 Deque

La **deque** (double-ended queue, coda a doppia estremità) è una struttura dati della STL che permette l'inserimento e la rimozione di elementi sia all'inizio sia alla fine del contenitore. Si comporta come una combinazione tra **vector** e **queue**: supporta accesso casuale agli elementi e operazioni efficienti alle estremità.

In figura 2.11 è illustrato il funzionamento logico di una **deque**.

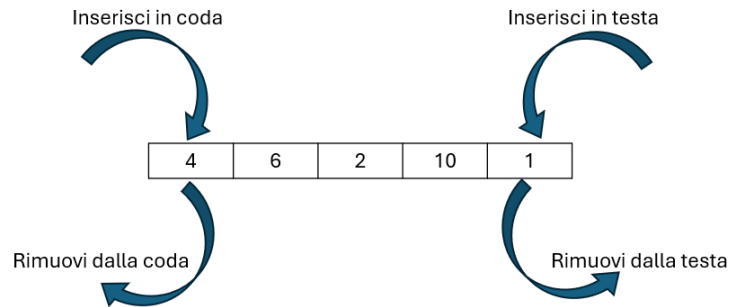


Figura 2.11: Rappresentazione grafica di una **deque**

2.8.1 Dichiarazione di una deque

```
1  #include <deque>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      deque<int> d;           // deque vuota di interi
7
8      d.push_back(10);        // inserisce 10 in coda
9      d.push_front(5);        // inserisce 5 in testa
10     d.push_back(20);         // inserisce 20 in coda
11
12     cout << "Front: " << d.front() << endl; // stampa 5
13     cout << "Back: " << d.back() << endl;   // stampa 20
14 }
```

Note:

- La **deque** permette inserimenti e cancellazioni sia in testa sia in coda in tempo costante $O(1)$.
- Supporta accesso casuale agli elementi tramite l'operatore `[]`, come un **vector**.
- È più flessibile di un **queue** o **stack** perché consente operazioni su entrambe le estremità.

2.8.2 Operazioni principali sulla deque

- **push_front(valore)** – Inserisce un elemento all'inizio.

```
1  d.push_front(3);
```

Complessità: $O(1)$

- **push_back(valore)** – Inserisce un elemento alla fine.

```
1 d.push_back(7);
```

Complessità: $O(1)$

- **pop_front()** – Rimuove l'elemento in testa.

```
1 d.pop_front();
```

Complessità: $O(1)$

- **pop_back()** – Rimuove l'elemento in coda.

```
1 d.pop_back();
```

Complessità: $O(1)$

- **front()** – Restituisce riferimento al primo elemento.

```
1 cout << d.front();
```

Complessità: $O(1)$

- **back()** – Restituisce riferimento all'ultimo elemento.

```
1 cout << d.back();
```

Complessità: $O(1)$

- **operator[]** – Accesso casuale ad un elemento.

```
1 cout << d[1]; // secondo elemento
```

Complessità: $O(1)$

- **size()** – Numero di elementi.

```
1 cout << d.size();
```

Complessità: $O(1)$

- **empty()** – Verifica se la deque è vuota.

```
1 if(d.empty()) cout << "Deque vuota!";
```

Complessità: $O(1)$

2.8.3 Iterazione su una deque

```
1 for(auto it = d.begin(); it != d.end(); ++it)
2   cout << *it << " ";
3
4 // oppure con range-based for
5 for(int x : d)
6   cout << x << " ";
```

2.8.4 Esempio pratico

```
1  #include <deque>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      deque<int> d;
7
8      d.push_back(10);
9      d.push_front(5);
10     d.push_back(20);
11
12     cout << "Elementi nella deque: ";
13     for(int x : d) cout << x << " ";
14
15     d.pop_front();
16     cout << "\nDopo pop_front: ";
17     for(int x : d) cout << x << " ";
18
19     d.pop_back();
20     cout << "\nDopo pop_back: ";
21     for(int x : d) cout << x << " ";
22
23     return 0;
24 }
```

Output:

```
Elementi nella deque: 5 10 20
Dopo pop_front: 10 20
Dopo pop_back: 10
```

2.8.5 Complessità computazionale riassunta

Operazione	Complessità
Inserimento in testa <code>push_front(x)</code>	$O(1)$
Inserimento in coda <code>push_back(x)</code>	$O(1)$
Rimozione in testa <code>pop_front()</code>	$O(1)$
Rimozione in coda <code>pop_back()</code>	$O(1)$
Accesso al primo elemento <code>front()</code>	$O(1)$
Accesso all'ultimo elemento <code>back()</code>	$O(1)$
Accesso casuale <code>operator[]</code>	$O(1)$
Controllo se vuota <code>empty()</code>	$O(1)$
Dimensione <code>size()</code>	$O(1)$

Tabella 2.8: Complessità operazioni principali sulla deque

2.8.6 Differenze tra queue e deque

A questo punto potremmo chiederci perché usare una **queue** quando abbiamo a disposizione la **deque** che, effettivamente, fa più cose senza aumentare la complessità computazionale.

La differenza principale tra **queue** e **deque**, aldilà delle funzionalità base, sta nel tipo di astrazione che vogliamo utilizzare.

- **queue**
 - È un **adattatore di contenitore**: ci interessa solo il comportamento FIFO.
 - Non permette l'accesso agli elementi centrali né l'iterazione.
 - L'interfaccia è semplice: `push`, `pop`, `front`, `back`.
 - Vantaggio: garantisce **chiarezza e sicurezza**, perché chi legge il codice sa subito che si sta usando una coda "pura".
- **deque**
 - È un **contenitore generico**: permette quasi tutte le operazioni, come iterazione, accesso casuale, inserimenti/rimozioni sia in testa che in coda.
 - Offre più flessibilità, ma anche più possibilità di usare la struttura in modo non coerente con una coda FIFO.

In pratica:

- Se vogliamo solo una coda, usiamo **queue**: comunica chiaramente l'intento e riduce il rischio di errori.

- Se vogliamo più flessibilità (accesso casuale, iterazione, manipolazioni complesse), usiamo deque.

È come scegliere tra un coltellino e un coltellino svizzero: il coltellino fa una cosa e la fa bene, il coltellino può fare di più ma non sempre è la scelta più chiara.

Capitolo 3

Brute Force e Algoritmi Greedy

In questo capitolo analizzeremo due approcci fondamentali alla risoluzione dei problemi: gli algoritmi **Brute Force** e gli algoritmi **Greedy**. Si tratta di due paradigmi concettualmente semplici, ma molto diversi tra loro per strategia, prestazioni e casi d'uso.

Il **Brute Force** adotta un approccio esaustivo: esplora tutte le soluzioni possibili e seleziona quella corretta o quella ottimale. È un metodo semplice, intuitivo e sempre corretto, ma spesso troppo lento per input di dimensioni elevate.

Gli algoritmi **Greedy**, al contrario, seguono una strategia “miope”: a ogni passo scelgono la soluzione che sembra migliore in quel momento, senza considerare le conseguenze future. Questo permette grande efficienza, ma non garantisce sempre una soluzione ottimale.

3.1 Algoritmi Brute Force

Gli algoritmi Brute Force rappresentano il metodo più semplice e diretto per risolvere un problema: consistono nel generare e valutare **tutte** le soluzioni possibili, scegliendo poi quella corretta o quella ottimale. Questo approccio è intuitivo e non richiede strutture dati o strategie sofisticate: si limita a esplorare in maniera esaustiva l'intero spazio delle soluzioni.

La forza del metodo Brute Force risiede nella sua semplicità: poiché vengono considerate tutte le possibilità, l'algoritmo garantisce sempre la correttezza della soluzione trovata. Tuttavia, la sua debolezza è altrettanto evidente: il numero di combinazioni cresce spesso in modo **esponenziale**, rendendo il Brute Force impraticabile per problemi di dimensioni medio-grandi.

Nonostante ciò, gli algoritmi Brute Force sono fondamentali per diversi motivi:

- sono facili da implementare e utili come punto di partenza;
- forniscono una soluzione ottima, quando è possibile enumerare tutto lo spazio delle soluzioni;
- permettono di confrontare la qualità di algoritmi più avanzati;
- sono spesso accettabili quando i dati in input sono piccoli oppure quando il problema non richiede una soluzione immediata.

3.1.1 Antivirus

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Il nuovo sistema di gara delle Selezioni Territoriali funziona alla grande, ma sembra che la mascotte abbia fiutato un virus nascosto fra i file inviati da un partecipante! Conosciamo la lunghezza del virus e sappiamo che si ripete uguale nei quattro file ricevuti, ma non sappiamo dove. Aiutaci a individuare il virus!

Dettagli:

Sono dati in input quattro file F_1, F_2, F_3, F_4 , rappresentati come quattro stringhe di caratteri di lunghezza rispettivamente N_1, N_2, N_3, N_4 .

Il virus è una stringa V di lunghezza M . La lunghezza M è nota (viene fornita in input), ma non conosciamo il contenuto della stringa V del virus. Sappiamo con certezza che V appare all'interno di tutti e quattro i file, come sottostringa di caratteri *consecutivi*. Sappiamo inoltre che *NON* ci sono altre sottostringhe consecutive di lunghezza M che si ripetono uguali in tutti e quattro i file.

Le posizioni dei caratteri nelle stringhe sono numerati a partire da 0. Per ciascuno dei quattro file F_i , trova la posizione in cui è inserito il virus, ovvero la posizione dove appare il primo carattere del virus V all'interno della stringa F_i .

L'obiettivo è: per ciascuno dei quattro file F_i , trovare la posizione (indice, a partire da 0) in cui compare V , ovvero l'indice del primo carattere di V dentro F_i .

Formato di input

La prima riga contiene un intero T , il numero di casi di test. Seguono T casi di test, ognuno preceduto da una riga vuota.

Per ciascun caso di test:

- Una riga con quattro interi N_1, N_2, N_3, N_4 — le lunghezze dei quattro file.
- Una riga con un intero M — la lunghezza del virus.
- Quattro righe contenenti le stringhe F_1, F_2, F_3, F_4 .

Formato di output

Per ogni caso di test, stampare:

Case #t: $p_1 p_2 p_3 p_4$

dove t è il numero del caso (a partire da 1), e p_i è la posizione (indice 0-based) della prima occorrenza della stringa virus V dentro F_i .

Assunzioni e Vincoli

- $T = 12$
- $2 \leq N_1, N_2, N_3, N_4 \leq 100$
- $2 \leq M \leq 20$
- $M \leq \min(N_1, N_2, N_3, N_4)$
- Tutti i caratteri dei file sono lettere minuscole dell'alfabeto inflese (dalla a alla z), NON sono presenti spazi.
- È garantito che il virus esiste ed è unico

Esempio

Input:

2

8 12 10 7

4

ananasso

associazione

tassonomia

massone

6 9 11 10

3

simone

ponessimo

milionesimo

cassonetto

Output:

Case #1: 4 0 1 1

Case #2: 3 1 4 4

Spiegazione:

Nel primo caso il virus è “asso”: **an**an**asso**, **ass**ociazione, **tass**onomia, **mass**one.

Nel secondo caso il virus è “one”: **sim**one, **pon**essimo, **milione**simo, **cassone**tto.

Idea (brute force):

Si considerano tutte le possibili sottostringhe di lunghezza M della prima stringa F_1 . Ogni sottostringa di questo tipo costituisce una possibile candidata a essere il virus.

Per ciascuna sottostringa candidata:

- si verifica se essa compare almeno una volta nella stringa F_2 ;
- si verifica se essa compare almeno una volta nella stringa F_3 ;
- si verifica se essa compare almeno una volta nella stringa F_4 .

Il controllo viene effettuato confrontando la sottostringa candidata con tutte le sottostringhe di lunghezza M delle altre stringhe.

Appena una sottostringa candidata risulta presente in tutte e quattro le stringhe, essa è necessariamente il virus, poiché il problema ne garantisce l'esistenza e l'unicità; a questo punto l'algoritmo può terminare.

Infine, si memorizzano le posizioni in cui il virus compare all'interno di ciascuna stringa.

Implementazione C++ - Brute Force

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      // Apro il file di input
7      ifstream inputFile("antivirus_input_1.txt");
8
9      int T;                                // Numero di test case nel file
10     inputFile >> T;
11
12     // Ciclo su tutti i test case
13     for (int test = 1; test <= T; ++test)
14     {
15         // N1, N2, N3, N4 sono le lunghezze delle quattro stringhe
16         // F1, F2, F3, F4
17         int N1, N2, N3, N4;
18         inputFile >> N1 >> N2 >> N3 >> N4;
19
20         // M la lunghezza della sottostringa del virus
21         int M;
22         inputFile >> M;
23
24         // Le quattro stringhe in cui cercare il virus
25         string F1, F2, F3, F4;
26         inputFile >> F1 >> F2 >> F3 >> F4;
27
28         // Variabili per salvare la posizione in cui il virus
29         // appare in ciascuna stringa
30         int F1_virus; // Posizione in F1 da cui inizia la
31         // sottostringa candidata
32         int F2_virus; // Posizione in F2 dove compare la
33         // sottostringa candidata
34         int F3_virus; // Posizione in F3 dove compare la
35         // sottostringa candidata
36         int F4_virus; // Posizione in F4 dove compare la
37         // sottostringa candidata
38
39         /*
40         Ciclo principale: scorriamo tutte le sottostringhe di
41         lunghezza M in F1
42         e le consideriamo candidate al virus
43         */
44         for(int i = 0; i <= F1.length(); i++){
45             F1_virus = i; // La sottostringa candidata parte da
46             // posizione i in F1
47             F2_virus = -1; // Inizializzo a -1, se rimane -1 vuol
48             // dire che non l'ho trovata
49             F3_virus = -1;
50             F4_virus = -1;
51
52             // Estraggo la sottostringa candidata dalla stringa F1
```

```

44         string virus_candidato = F1.substr(i, M);
45
46         // Cerco il virus in F2
47         for(int j = 0; j < F2.length(); j++){
48             if(F2.substr(j, M) == virus_candidato){ //
49                 confronto ogni sottostringa di F2
50                 F2_virus = j; // trovato, salvo la posizione
51                 break; // esco dal ciclo
52             }
53         }
54
55         // Cerco il virus in F3
56         for(int j = 0; j < F3.length(); j++){
57             if(F3.substr(j, M) == virus_candidato){
58                 F3_virus = j;
59                 break;
60             }
61         }
62
63         // Cerco il virus in F4
64         for(int j = 0; j < F4.length(); j++){
65             if(F4.substr(j, M) == virus_candidato){
66                 F4_virus = j;
67                 break;
68             }
69         }
70
71         // Se la sottostringa candidata e' presente in tutte e
72         // quattro le stringhe, abbiamo trovato il virus
73         if(F2_virus != -1 && F3_virus != -1 && F4_virus != -1){
74             break; // esco dal ciclo su F1
75         }
76
77         // Stampo le posizioni trovate per ciascuna stringa
78         cout << "Case #" << test << ": "
79         << F1_virus << " "
80         << F2_virus << " "
81         << F3_virus << " "
82         << F4_virus << endl;
83     }
84
85     inputFile.close(); // Chiudo il file di input
86     return 0;
87 }

```

Costo Computazionale:

Il costo computazionale è:

- $O(N_2) + O(N_3) + O(N_4)$ per i cicli interni nelle righe 47, 55, 63;
- $O(N_1)$ per il ciclo esterno alla riga 37.

Ciò significa che i cicli più interni vengono eseguiti N_1 volte.
La complessità totale è quindi:

$$\begin{aligned} &= O(N_1) \cdot [O(N_2) + O(N_3) + O(N_4)] \\ &\approx O(N_1) \cdot \max(O(N_2), O(N_3), O(N_4)) \end{aligned}$$

Dai vincoli del problema:

$$2 \leq N_i \leq 100$$

la complessità può essere stimata come:

$$\begin{aligned} &\approx O(N_1) \cdot O(100) \\ &\approx O(100 \cdot 100) \\ &= O(10^4) \end{aligned}$$

Pertanto, considerando una macchina che esegue circa 10^8 operazioni al secondo, il tempo stimato di esecuzione sarebbe:

$$\frac{10^4}{10^8} \text{ secondi} = 10^{-4} \text{ secondi.}$$

In conclusione, il programma è coerente con il limite di tempo imposto dal problema (1.00 s).

L'approccio *brute force* è, in generale, il più dispendioso dal punto di vista computazionale. In questo caso specifico, tuttavia, la dimensione ridotta degli input consente di rientrare nei limiti temporali previsti, permettendo al programma di terminare correttamente l'esecuzione.

Nella maggior parte dei casi, però, tale strategia non è applicabile e si rende necessario ricorrere a metodi algoritmici più efficienti.

Di seguito vengono presentati due problemi risolti mediante un approccio *brute force* che, nonostante la correttezza logica della soluzione, non rispettano i vincoli temporali imposti dal problema.

3.1.2 Stick Lengths

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

There are n sticks with some lengths. Your task is to modify the sticks so that each stick has the same length.

You can either lengthen and shorten each stick. Both operations cost x where x is the difference between the new and original length.

What is the minimum total cost?

Input:

The first input line contains an integer n : the number of sticks.

Then there are n integers: $p_1, p_2, \dots, p_n$: the lengths of the sticks.

Output:

Print one integer: the minimum total cost.

Constraints:

$$1 \leq n \leq 2 \cdot 10^5$$

$$1 \leq p_i \leq 10^9$$

Example:

Input:

```
5
2 3 1 5 2
```

Output:

```
5
```

Idea (brute force):

provare ogni possibile scelta di lunghezza target tra quelle presenti nei dati e calcolare il costo totale per uniformare tutti i bastoncini a quel valore; quindi scegliere il minimo tra tutti i costi calcolati. Nel caso d'esempio d'input, occorre calcolare quanto costa uniformare tutti i numeri a 1, poi a 2, poi a 3, poi a 4 ed infine a 5

Implementazione C++ - Brute Force:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     vector<int> sticks;           // Vettore che conterra' le lunghezze
        dei bastoncini
7     int numbers;                 // Numero di bastoncini n
8     int costo_minimo = INT_MAX;  // Variabile per memorizzare il costo
        minimo trovato, inizializzata all'infinito
9
10    cin >> numbers;              // Leggo n, il numero di bastoncini
```

```

11
12 // Ciclo per leggere le lunghezze dei bastoncini:  $O(n)$ 
13 for (int i = 0; i < numbers; i++)
14 {
15     int stick;
16     cin >> stick;                // Leggo la lunghezza di un
        bastoncino
17     sticks.push_back(stick);    // Inserisco la lunghezza nel vettore
18 }
19
20 // Calcolo il minimo e il massimo tra le lunghezze:  $O(n)$ 
21 int minimo = *min_element(sticks.begin(), sticks.end());
22 int massimo = *max_element(sticks.begin(), sticks.end());
23
24 // Ciclo principale: provo ogni possibile lunghezza target tra
    minimo e massimo
25 // Complessita':  $O(R * n)$ , dove  $R = \text{massimo} - \text{minimo}$ 
26 for (int i = minimo; i <= massimo; i++)
27 {
28     int somma_costi = 0; // Somma dei costi per portare tutti i
        bastoncini alla lunghezza target i
29
30     // Per ogni bastoncino calcolo il costo per portarlo alla
        lunghezza target i:  $O(n)$ 
31     for (int stick : sticks)
32     {
33         int costo = abs(stick - i); // Differenza tra lunghezza
            attuale e target
34         somma_costi += costo;        // Aggiungo il costo totale
35     }
36
37     // Aggiorno il costo minimo se la somma dei costi corrente
        inferiore
38     costo_minimo = min(costo_minimo, somma_costi);
39 }
40
41 cout << costo_minimo; // Stampo il costo totale minimo
42
43 return 0;
44 }

```

Costo Computazionale:

Il costo computazionale è:

- $O(n)$ per il for della riga 13
- $O(n)$ per la funzione *min_element* della riga 21
- $O(n)$ per la funzione *max_element* della riga 22
- $O(\text{massimo} - \text{minimo}) \cdot O(n)$ per il for della riga 26 (incluso l'interno)

La complessità è:

$$\begin{aligned} &= O(n) + O(n) + O(n) + O(\text{massimo} - \text{minimo}) \cdot O(n) \\ &= O(n) + O(R \cdot n) \\ &= O(R \cdot n) \end{aligned}$$

dove: $R = \text{massimo} - \text{minimo}$, $n = \text{numero di bastoncini}$

Dai vincoli del problema:

$$1 \leq n \leq 2 \cdot 10^5, \quad 1 \leq p_i \leq 10^9,$$

quindi il valore massimo di R può arrivare fino a

$$R \leq 10^9.$$

Sostituendo nella complessità, otteniamo:

$$O(R \cdot n) = O(10^9 \cdot 2 \cdot 10^5) = O(2 \cdot 10^{14}).$$

Considerando una macchina che esegue circa 10^8 operazioni al secondo, il tempo stimato di esecuzione sarebbe:

$$\frac{2 \cdot 10^{14}}{10^8} = 2 \cdot 10^6 \text{ secondi} \approx 23 \text{ giorni!}$$

Pertanto, questo approccio è praticamente inefficiente e supera di gran lunga il time limit di 1 secondo.

Per risolvere il problema in tempo ragionevole, è necessario un algoritmo più efficiente, ad esempio basato sulla mediana dei bastoncini che vedremo in seguito.

3.1.3 Nearest Smaller Values

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Given an array of n integers, your task is to find for each array position the nearest position to its left having a smaller value.

Input: The first input line has an integer n : the size of the array. The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Output: Print n integers: for each array position the nearest position with a smaller value. If there is no such position, print 0.

Constraints:

$$1 \leq n \leq 2 \cdot 10^5$$

$$1 \leq x_i \leq 10^9$$

Example:

Input:

8

2 5 1 4 8 3 2 5

Output:

0 1 0 3 4 3 3 7

Idea (brute force): Dato un array di N numeri interi, per ogni posizione i si vogliono analizzare tutti gli elementi che si trovano alla sua sinistra (cioè con indice minore di i) per individuare un valore strettamente più piccolo di quello corrente. Tra tutti i valori più piccoli trovati, si seleziona quello più vicino alla posizione i , cioè quello con l'indice maggiore tra quelli validi.

Il programma utilizza due cicli annidati:

- il ciclo esterno scorre tutte le posizioni dell'array;
- il ciclo interno scorre le posizioni precedenti a quella corrente, confrontando i valori.

Ogni volta che viene trovato un valore più piccolo, la sua posizione viene salvata come possibile risposta. Poiché il ciclo interno procede da sinistra verso destra, l'ultima posizione salvata corrisponde alla più vicina a sinistra, che è quella richiesta dal problema.

Se per una determinata posizione non viene trovato alcun valore più piccolo, il risultato rimane pari a 0, indicando che non esiste una posizione valida alla sua sinistra.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int64_t N;          // Numero di elementi dell'array
7      cin >> N;          // Lettura di N da input
8
9      // array      : contiene i valori dell'array
10     // results    : contiene la risposta per ogni posizione,
11     //              inizializzata a 0 (nessun elemento pi piccolo a
12     //              sinistra)
13
14     vector<int> array, results(N, 0);
15
16     // Lettura dei valori dell'array
17     for (int i = 0; i < N; i++)
18     {
19         int64_t x;        // Variabile temporanea per leggere ogni valore
20         cin >> x;        // Lettura del valore
21         array.push_back(x); // Inserimento in coda al vector
22     }
23
24     // Per ogni posizione i dell'array
25     for (int i = 0; i < N; i++)
26     {
27         // Scorriamo tutti gli elementi a sinistra di i
28         for (int j = 0; j < i; j++)
29         {
30             // Se troviamo un valore pi piccolo di array[i]
31             if (array[j] < array[i])
32             {
33                 // Aggiorniamo la risposta con la posizione (1-based)
34                 /* IMPORTANTE: Non interrompiamo il ciclo. alla fine
35                 rimarrà
36                 l'ULTIMA posizione valida trovata (la pi vicina a i)
37                 */
38                 results[i] = j + 1;
39             }
40         }
41     }
42
43     // Stampa delle soluzioni
44     for (int result : results)
45     {
46         cout << result << " ";
47     }
48
49     return 0;
50 }

```

Costo Computazionale:

Il costo computazionale di questo programma può essere analizzato come segue:

- $O(N)$ per il ciclo for della riga 16.
- $O(i) = O(N - 1)$ per il ciclo for della riga 27 (la i arriva fino a $N - 1$).

Il costo computazionale è quindi:

$$= O(N) \cdot O(N - 1) = O(N) \cdot O(N) = O(N^2)$$

Questo significa che il numero di operazioni cresce quadraticamente con la dimensione dell'array. Considerando però che, dai vincoli del problema, $N \leq 2 \cdot 10^5$, allora quando N diventa troppo grande si ha:

$$O((10^5)^2) = O(10^{10})$$

che è superiore alle 10^8 operazioni che la nostra macchina può fare in 1 secondo. Occorre quindi trovare una strategia diversa per poter risolvere il problema nei limiti imposti.

3.2 Idee Greedy

Gli algoritmi *greedy* si basano sull'idea di compiere, a ogni passo, la scelta che risulta immediatamente più conveniente, senza considerare in modo esplicito le conseguenze future. La soluzione viene quindi costruita progressivamente, fissando le decisioni man mano che vengono prese e senza possibilità di modificarle successivamente.

Questo approccio consente spesso di ottenere algoritmi semplici ed efficienti, caratterizzati da una bassa complessità computazionale. Tuttavia, la sua applicabilità non è universale: una soluzione greedy è corretta solo se il problema ammette una struttura tale per cui una scelta ottimale a livello locale conduce anche a una soluzione ottimale globale. In assenza di questa proprietà, l'algoritmo può produrre risultati non corretti, pur risultando computazionalmente rapido.

In questo contesto, l'analisi dei controesempi riveste un ruolo fondamentale. Un singolo controesempio è infatti sufficiente a dimostrare la non correttezza di una strategia greedy, mostrando come una scelta localmente ottimale possa condurre a una soluzione globale subottimale. Per questo motivo, è importante imparare a costruire esempi che mostrino come un algoritmo greedy possa fallire, evidenziandone così la non validità.

Esempio di idea Greedy:

Supponiamo di avere una rete di piazze collegate tra loro da strade. In ogni piazza vi è un certo quantitativo di monete a terra che possono essere raccolte solo se si passa per quella determinata piazza.

Bisogna costruire un algoritmo che determini il percorso che permetta di raccogliere il maggior numero di monete.

Con riferimento alla figura 3.1, partendo dalla piazza M_1 , posso scegliere tra due percorsi:

- $M_1 \rightarrow M_2 \rightarrow M_3$
- $M_1 \rightarrow M_4 \rightarrow M_5$

Qual è il percorso più redditizio?

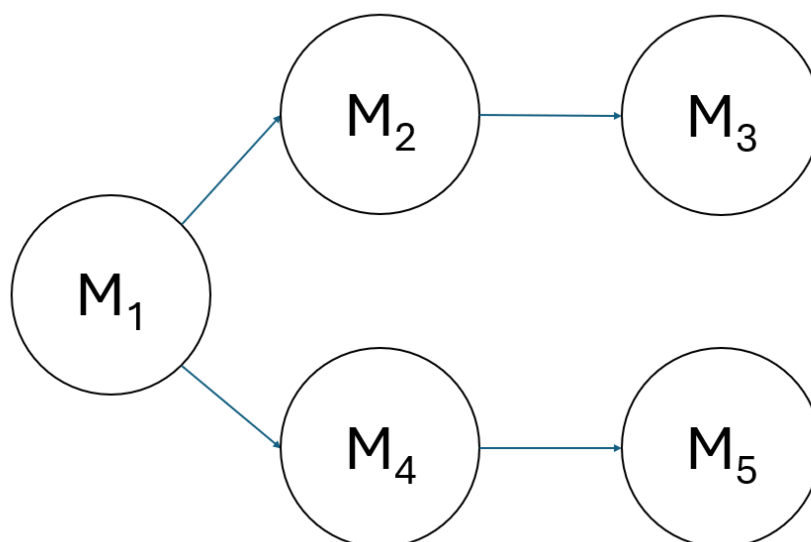


Figura 3.1: Percorso con piazze (cerchi) e strade (freccie). Con M_i sono indicate le monete presenti in ogni piazza

L'approccio **Brute Force** consiste nel percorrere tutte le strade e, quindi, valutare il massimo tra:

- $M_1 + M_2 + M_3$
- $M_1 + M_4 + M_5$

Ciò garantisce la correttezza della soluzione pagando in termini di complessità computazionale: al crescere delle possibili strade, l'algoritmo diventa insostenibile in termini di costo.

Un approccio **Greedy**, invece, consiste nel prendere, ad ogni piazza, la scelta immediatamente più conveniente, cioè seguire la strada che conduce alla piazza con il maggior numero di monete tra quelle disponibili. In altre parole, a ogni nodo del grafo, si seleziona localmente il percorso che massimizza la quantità di monete raccolte in quel passo, senza considerare tutte le possibili combinazioni future.

Con riferimento alla figura 3.1, partendo dalla piazza M_1 , la scelta greedy consiste nel confrontare le piazze raggiungibili (M_2 e M_4) e scegliere immediatamente quella con il maggior numero di monete. Successivamente, si ripete lo stesso criterio per la piazza successiva fino a raggiungere una piazza terminale o senza uscite disponibili.

Come detto, l'approccio si concretizza nel fare la scelta migliore *localmente*, senza considerare le conseguenze future.

Questo metodo è computazionalmente efficiente, in quanto richiede solo di confrontare i valori locali delle piazze accessibili in ogni passo. Tuttavia, *non sempre garantisce il massimo globale*, poiché una scelta ottimale locale potrebbe portare a un percorso complessivamente subottimale. Per questo motivo, la strategia greedy va applicata solo quando il problema possiede una struttura che assicuri la correttezza delle scelte locali rispetto alla soluzione globale, oppure come heuristica rapida in problemi di grandi dimensioni. A tal proposito, in figura 3.2 è riportato un controesempio che smonta la scelta greedy sopra proposta.

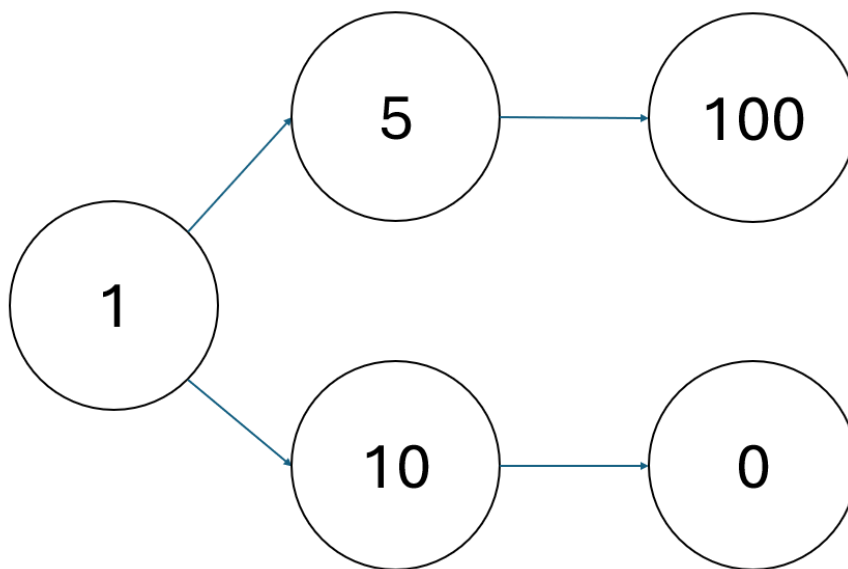


Figura 3.2: Controesempio: scegliendo al primo passo la piazza che garantisce il massimo immediato (10 monete), il percorso totale raccoglie solo 11 monete, mentre scegliendo l'altra strada si potrebbero raccogliere 106 monete. Questo evidenzia come la scelta greedy locale possa portare a un risultato subottimale.

3.2.1 Calcolatrice Rotta

[Link al problema](#)

Francesco ha fatto cadere la sua calcolatrice, e ora non funziona più come dovrebbe! Gli unici tasti funzionanti sono il $-$, il $\times$, l'1 e il 2.

Per utilizzare la calcolatrice è costretto a partire dal numero 1 o dal numero 2 (premendo il tasto corrispondente) e applicare zero o più volte una delle 4 possibili operazioni funzionanti:

- sottrarre 1
- sottrarre 2
- moltiplicare per 1
- moltiplicare per 2

Per fare uno scherzo ai suoi amici, vorrebbe raggiungere sulla calcolatrice il numero N . Quante operazioni deve fare al minimo per farlo?

Nota: *premere il tasto 1 o 2 all'inizio conta come un'operazione, inoltre la calcolatrice è danneggiata quindi Francesco è costretto a partire sempre da 1 o 2 (non può ad esempio premere 2 e poi 1 per partire dal numero 21) e le uniche operazioni ammesse sono quelle precedentemente elencate (non può per esempio moltiplicare o sottrarre 12 o 21).*

3.2.2 Sticks Length

Link al problema

Time limit = 1.00s, Memory limit = 512 MB

Ci sono n bastoncini con determinate lunghezze. Il tuo compito è modificare i bastoncini in modo che tutti abbiano la stessa lunghezza.

Puoi sia allungare che accorciare ogni bastoncino. Entrambe le operazioni hanno un costo pari a x , dove x è la differenza tra la nuova lunghezza e quella originale.

L'obiettivo è determinare qual è il costo totale minimo necessario per rendere tutti i bastoncini della stessa lunghezza.

Formato input:

La prima riga contiene un intero n , il numero di bastoncini. La seconda riga contiene n interi $p_1, p_2, \dots, p_n$, le lunghezze dei bastoncini.

Formato output:

Stampa un intero: il costo totale minimo.

Assunzioni e Vincoli:

$$1 \leq n \leq 2 \cdot 10^5$$

$$1 \leq p_i \leq 10^9$$

Esempio:

Input:

5

2 3 1 5 2

Output:

5

Idea (greedy):

Rendere tutti i bastoncini di una stessa lunghezza L comporta un costo pari alla somma delle variazioni necessarie, cioè

$$\begin{aligned} \text{Costo} &= |p_1 - L| + |p_2 - L| + \cdots + |p_n - L| \\ &= \sum_{i=1}^n |p_i - L|. \end{aligned}$$

Se L viene scelto troppo piccolo, i bastoncini più lunghi dovranno essere accorciati di parecchio; al contrario, se L è troppo grande, molti bastoncini dovranno essere allungati. L'obiettivo è quindi bilanciare queste due operazioni opposte.

Ordinando le lunghezze, scegliere la *mediana* garantisce che almeno metà dei bastoncini abbia lunghezza minore o uguale a L e almeno metà maggiore o uguale a L . In questo modo gli incrementi e le riduzioni si compensano nel miglior modo possibile, evitando spostamenti inutili che farebbero crescere il costo totale.

Per questo motivo la strategia greedy ottimale consiste nel fissare come lunghezza finale la mediana delle lunghezze iniziali e calcolare il costo totale come somma delle differenze assolute.

Appendice A

Regolamento 2025/2026

Ai Direttori
degli Uffici Scolastici Regionali
LORO SEDI

Ai Dirigenti scolastici
degli Istituti secondari superiori statali
LORO SEDI

Ai Dirigenti scolastici
degli Istituti secondari superiori paritari
LORO SEDI

Campionati Italiani di Informatica (ex Olimpiadi) edizione 2025-2026

I Campionati Italiani di Informatica (ex Olimpiadi) sono una competizione rivolta agli studenti delle istituzioni scolastiche secondarie di II grado. Giunte ormai alla XXVI edizione, fanno parte del programma di valorizzazione delle eccellenze che la Direzione Generale per gli ordinamenti scolastici e la valutazione del sistema nazionale di istruzione del Ministero dell'Istruzione promuove e finanzia ogni anno. Inoltre, in virtù del Protocollo di intesa tra Ministero dell'Istruzione e l'Associazione Italiana per l'Informatica e il Calcolo Automatico (AICA) del 31 luglio 2023, la competizione viene promossa con il supporto tecnico e logistico di AICA, e si avvale del supporto amministrativo contabile dell'ITE E. Tosi di Busto Arsizio (VA).

Rispetto alla precedente edizione ci saranno i seguenti principali cambiamenti, in breve:

- **resa facoltativa l'iscrizione degli studenti per la fase scolastica;**
- **aggiunta di una sessione di prova prima della fase scolastica;**
- **possibilità di spalmare i turni della fase scolastica su due giorni;**
- **rimosso il requisito di AlgoBadge prima delle territoriali;**
- **definizione degli ammissibili alle nazionali subito dopo le territoriali;**
- **possibilità di partecipazione alle nazionali in modalità online per gli ammissibili non ammessi;**
- **conferimento delle menzioni d'onore alla fase nazionale.**

L'evento costituisce occasione per far emergere e valorizzare le eccellenze esistenti nella scuola italiana, con positiva ricaduta sull'intero sistema educativo. Inoltre, la manifestazione assume particolare rilevanza per i significativi riconoscimenti ricevuti, tra cui l'assegnazione di Borse di Studio da parte della Banca d'Italia per stage all'estero ai migliori classificati.

Si auspica pertanto che, dopo le costanti e brillanti affermazioni delle precedenti edizioni, l'Italia possa partecipare alle prossime *International Olympiad in Informatics (IOI)* con i suoi migliori studenti e che gli stessi docenti traggano dalle OII elementi utili a migliorare le loro tecniche di insegnamento. A livello territoriale, verranno promosse attività di formazione tese a migliorare la preparazione degli studenti in vista delle selezioni territoriali e nazionali.

La partecipazione è aperta alle classi I-IV di tutte le istituzioni scolastiche secondarie di II grado, statali e paritarie, che ritengano di avere studenti con potenziale interesse per l'informatica, soprattutto riguardo gli aspetti logici e algoritmici di tale disciplina. La partecipazione è adatta e consigliata anche a coloro che ancora non hanno studiato informatica a livello curricolare, per consentire agli studenti di interessarsi e avvicinarsi gradualmente alla materia.

1. Modalità di iscrizione

- Le scuole che intendono aderire devono iscriversi registrando i dati necessari tramite il portale [olimpiadi-scientifiche.it](https://www.olimpiadi-scientifiche.it). L'iscrizione dell'istituzione avviene tramite il docente Referente Scolastico, cioè la persona con cui l'organizzazione olimpica si terrà in contatto per inviare tutte le informazioni e le comunicazioni necessarie. I rapporti si terranno prevalentemente via e-mail. **Le iscrizioni degli Istituti sono già aperte e si chiuderanno giovedì 27 novembre 2025 alle ore 23:59.**
- Il referente deve quindi seguire questi tre passaggi: (1) creare un utente di "olimpiadi-scientifiche" e verificare la mail associata; (2) associarsi alla propria scuola come insegnante; (3) cliccare su "iscriviti" alle "Olimpiadi Italiane di Informatica 2025/2026". Se il referente possiede già un account insegnante di "olimpiadi-scientifiche" è sufficiente il solo passaggio (3). Alcuni video-tutorial sulla gestione delle iscrizioni per gli insegnanti sono disponibili qui: <https://www.youtube.com/playlist?list=PLO4y9a8ITpK0PXwYX0y5cPT4mIA-k1iME>
- Una volta che la scuola sarà iscritta, anche gli studenti di quella scuola potranno iscriversi alla prima fase scolastica della competizione sempre tramite il medesimo portale. Le iscrizioni degli studenti saranno soggette ad approvazione del referente scolastico che controllerà il rispetto dei requisiti di ammissione, e il consenso dei genitori nel caso di studenti di età inferiore a 14 anni. L'iscrizione degli studenti si chiuderà **lunedì 12 gennaio 2026 alle ore 23:59**, e non sarà obbligatoria per partecipare alla fase scolastica, mentre sarà obbligatoria per partecipare alla fase territoriale.
- Organizzare le Olimpiadi di Informatica richiede numerose risorse, sia in termini di persone che economici. Se ritenete importante offrire ai ragazzi la possibilità di partecipare a questo evento, e per dare sostegno allo svolgimento delle nostre attività, vi invitiamo—se ne avete la possibilità—a effettuare una donazione **libera e facoltativa**, con importo consigliato di 80€, tramite bonifico a:

Destinatario: Associazione Nazionale Programmazione Competitiva APS

IBAN: IT69S0623033130000047146755

BIC: CRPPIT2P338

Causale: Donazione per le OII da <codice meccanografico della scuola>

Il contributo è libero e facoltativo e non farlo non preclude la partecipazione né ha alcun effetto negativo sul regolare svolgimento dell'iniziativa nella vostra scuola.

2. Preparazione alla fase scolastica

Per essere ammissibile alla selezione scolastica uno studente deve soddisfare tutte le seguenti condizioni:

- Essere iscritto a una delle prime quattro classi dei corsi quinquennali (o delle prime tre classi dei corsi quadriennali, laddove previsto) di una scuola secondaria di II grado iscritta alla competizione. Uno studente si può anche appoggiare per l'iscrizione a una scuola vicina iscritta, con il benestare dalla scuola di appartenenza (nel caso in cui quest'ultima non fosse iscritta).
- Essere nato dopo il 30 giugno 2007.
- Impegnarsi a tenere un comportamento etico durante tutte le fasi della competizione, senza cercare di copiare o aggirare il funzionamento dei sistemi di gara in alcun modo; non avendo contenziosi in corso non ancora risolti con i Campionati Italiani di Informatica.
- Impegnarsi, qualora superi le prime prove di selezione, a frequentare i corsi di formazione che si terranno prima della competizione internazionale.
- Impegnarsi, qualora superi l'ultima selezione, a recarsi all'estero nel periodo delle gare internazionali con gli accompagnatori designati dal Comitato.

Gli studenti potranno prepararsi alla prima fase tramite le prove degli anni passati disponibili sul sito [scolastiche.olinfo.it](https://www.scolastiche.olinfo.it). Inoltre, **differentemente dagli scorsi anni, prepareremo una sessione di prova** con problemi tratti dalle scorse edizioni, che consigliamo di proporre agli studenti nella settimana **dall'1 al 5 dicembre**. La sessione avrà lo stesso formato, argomenti e livello di difficoltà della prova effettiva (descritto nella prossima sezione), e avrà il duplice scopo di essere un utile allenamento per gli studenti, e uno importante strumento per i docenti per assicurarsi di non incappare in problemi tecnici in seguito.

3. Prima fase: selezione scolastica (11-12 dicembre 2025)

La prova si svolgerà nei giorni **11 e 12 dicembre** nelle singole scuole e avrà la durata di **90 minuti**. Ogni scuola potrà scegliere autonomamente quando svolgere la prova, eventualmente su più turni al giorno o più giorni (**solo se necessario** per consentire la partecipazione a tutti gli studenti), con orari di inizio compresi tra le **ore 8:30 e le ore 16:30** di ciascun giorno. La prova potrà essere somministrata agli studenti tramite appropriati strumenti online, accessibili sia tramite computer che tramite tablet o smartphone. In caso siano previsti più turni tramite la piattaforma online, **questi non potranno sovrapporsi**. Gli studenti già registrati su olimpiadi-scientifiche.it potranno entrare nella piattaforma di gara con le loro credenziali. Inoltre, l'insegnante potrà inserire sul momento studenti a piacimento (registrati o no) inserendo per ciascuno i dati identificativi necessari (nome, cognome, classe). L'insegnante potrà visionare e modificare le prove dei suoi studenti durante e dopo la prova. **Sarà anche possibile effettuare la prova in modalità totalmente cartacea**: in tal caso il docente potrà accedere alla piattaforma di gestione per scaricare i testi dalle **ore 17:00 di mercoledì 10 dicembre**, e dovrà farsi carico dell'inserimento delle risposte di tutti i suoi studenti online entro le **ore 23:59 del 15 dicembre 2025**. Istruzioni dettagliate sulla piattaforma di gara per studenti e di gestione per insegnanti verranno inviate a ogni referente il giorno prima della gara.

La prova sarà preparata a livello nazionale dall'unità operativa tecnico-didattica del Comitato Olimpico. La prova consiste nella soluzione di 10 problemi a carattere logico-matematico, algoritmico e di programmazione in pseudo-codice (la cui descrizione è pubblicamente disponibile a [questo link](#)), ciascuno con due domande. Tutti i quesiti sono pensati per **non richiedere la conoscenza di alcun linguaggio di programmazione**, e la prova sarà pensata per **poter essere proposta a intere classi**, includendo un alto numero di quesiti semplici, al fine di risultare un'esperienza didattica utile anche per gli studenti meno preparati.

I quesiti saranno di tre tipologie: a risposta singola, per cui lo studente deve scegliere una tra cinque risposte proposte; a risposta multipla, per cui lo studente deve scegliere **tutte e sole** le opzioni corrette tra alcune proposte; o a risposta aperta, per cui è richiesto che una soluzione alfanumerica venga riportata liberamente dallo studente. La valutazione dei quesiti verrà effettuata come segue:

- a ogni risposta esatta vengono assegnati 5 punti;
- a ogni risposta sbagliata vengono assegnati 0 punti;
- ad una risposta vuota viene assegnato 1 punto se la domanda è a risposta singola, oppure 0 punti se la domanda è a risposta multipla o aperta.

È vietato consultare testi, manuali o appunti di qualsiasi genere, come pure accedere ad altre risorse informatiche al di fuori di quelle necessarie per sostenere la prova, pena l'esclusione dalla stessa.

4. Risultati fase scolastica e preparazione alla fase territoriale

Entro il 17 dicembre 2025, i risultati degli studenti nella fase scolastica saranno pubblicati online. Inoltre, sarà pubblicata una *quota* per ogni scuola, pari a 1 ogni 20 studenti partecipanti alla fase scolastica, arrotondata all'intero più vicino (la quota sarà quindi pari a zero in caso di meno di 10 partecipanti). Gli studenti partecipanti saranno quindi invitati a registrarsi su olimpiadi-scientifiche.it se non l'avessero ancora fatto, e a iniziare il percorso [AlgoBadge](#) (facendo login tramite il proprio account di olimpiadi-scientifiche) che sarà poi richiesto completare per accedere alla finale nazionale. Inoltre, da metà gennaio fino a domenica 22 febbraio sarà proposta agli studenti una gara di pratica, con problemi tratti dalle scorse edizioni, per familiarizzare con la piattaforma di gara che dovranno usare durante la selezione territoriale. **Saranno ammissibili alla selezione territoriale solo gli studenti che otterranno almeno 50 punti nella gara di pratica.**

Entro mercoledì 25 febbraio verrà quindi pubblicato l'elenco finale degli ammessi alla selezione territoriale. Saranno ammessi, tra gli studenti iscritti che hanno ottenuto almeno 50 punti nella gara di pratica:

- I primi studenti per punteggio di ogni scuola fino al raggiungimento della *quota* della scuola, favorendo in caso di pari merito gli studenti provenienti da anni di corso inferiore e più giovani, purché abbiano ottenuto un punteggio superiore a quello di una prova in bianco.

- Inoltre il Comitato Olimpico, a suo insindacabile giudizio, ammetterà alla selezione territoriale i migliori studenti in classifica a livello nazionale, che abbiano ottenuto un punteggio minimo determinato dal comitato. Sarà previsto un punteggio minimo differenziato per gli studenti del biennio. Potrebbe essere previsto un numero massimo di ammessi per scuola, qualora fosse ritenuto necessario dal comitato.
- Infine, saranno ammessi di diritto anche gli studenti iscritti alle OII con 50 punti nella gara di pratica che siano membri di una squadra ammessa alla finale delle Olimpiadi di Informatica a Squadre 2025/26, e gli studenti che abbiano partecipato in presenza o online alla selezione nazionale 2024/25 ottenendo una medaglia o menzione d'onore.
- Non parteciperanno invece alla selezione territoriale i PO 2025/26, in quanto già ammessi di diritto alla selezione nazionale.

Nei giorni seguenti, il Referente Territoriale effettuerà l'assegnazione delle sedi effettive di gara agli studenti, ad una distanza congrua da loro. Il Referente Territoriale può anche consentire agli studenti la partecipazione remota da una postazione a loro scelta, anche utilizzando un dispositivo personale ma con proctoring personale online, nel solo caso di comprovate difficoltà logistiche o sanitarie. L'assegnazione sarà completata entro martedì 24 marzo tramite olimpiadi-scientifiche, ed entro il 30 marzo saranno quindi comunicate agli studenti le sedi assegnate e relative istruzioni logistiche.

5. Seconda fase: selezione territoriale (15 aprile 2026)

La selezione è prevista avere inizio per tutti i partecipanti **alle ore 14:00 di mercoledì 15 aprile**, per una durata di 3 ore. I partecipanti dovranno recarsi nella sede di gara a loro assegnata, per partecipare ad un evento di selezione, formazione e networking organizzato autonomamente dalla Sede Territoriale locale nel corso della giornata. L'evento comprenderà in ogni caso una gara di selezione, preparata a livello nazionale dall'unità operativa tecnico-didattica del Comitato Olimpico e somministrata tramite strumenti online.

Ai referenti territoriali verranno consegnate le credenziali di accesso dei partecipanti della loro sede al sistema di gara, e saranno responsabili di distribuirle agli studenti. **Le credenziali sono personali e non trasferibili ad altri soggetti; coloro che le trasferiscono ad altri soggetti incorrono nelle sanzioni previste dall'attuale normativa in materia di frode e sono esclusi dalla gara.** Inoltre, testi e soluzioni dei problemi non possono essere in alcun modo condivisi o commentati fino al termine della prova, pena squalifica. È vietato accedere a risorse online (ad es. motori di ricerca e IA), consultare libri o appunti o altre risorse come LLM disponibili in locale, chiedere aiuto sulla risoluzione dei problemi e comunicare con chiunque, pena l'esclusione dalla gara. Tuttavia è possibile chiedere allo staff chiarimenti sul testo tramite il sistema di gara. **Controlli automatizzati verranno effettuati per individuare eventuali violazioni alle precedenti norme.**

La gara consisterà nella risoluzione di quattro problemi algoritmici tramite scrittura di programmi al computer, in un linguaggio di programmazione a scelta dello studente tra i linguaggi supportati:

- C
- C#
- C++
- Go
- Java
- JavaScript (con template in HTML)
- Pascal
- Python 3
- Visual Basic

Per questi linguaggi saranno disponibili dei template di soluzione con il codice relativo alla lettura dell'input e alla scrittura dell'output. Sarà possibile proporre l'uso di un altro linguaggio debitamente motivato, che potrà essere ammesso a giudizio insindacabile della commissione di gara.

I problemi valgono 50 punti ciascuno e verranno presentati in un ordine indicativo crescente del livello di difficoltà e conoscenze richieste dal problema. I partecipanti possono scegliere quali problemi risolvere e in quale ordine. La descrizione di ogni problema comprenderà esempi e limiti e/o condizioni dei dati di ingresso,

incluse eventuali condizioni su alcuni gruppi di casi di test. Ciascun problema va risolto in queste fasi:

Fase 1: richiesta dell'input

Si accede alla pagina del problema e si clicca sul pulsante Richiedi input. Ogni volta che viene premuto su Richiedi input viene generato un nuovo file di input, diverso dai precedenti e diverso da quello degli altri partecipanti. Dal momento della richiesta partirà un conto alla rovescia di **5 minuti** per completare il processo di sottoposizione.

Fase 2: download dell'input

A questo punto è disponibile per il download dal browser (cliccando su Scarica input) il file di input generato: il nome del file scaricato sarà *nomebreve_input_N.txt* dove al posto di *N* ci sarà un numero progressivo (a partire da 1). Il file conterrà caratteri di fine riga in stile Unix (singolo carattere '\n'), potrebbe essere necessario usare programmi specifici per visualizzare correttamente il file. È consentito scaricare nuovamente il file qualora fosse necessario;

Fase 3: calcolo dell'output

L'input appena scaricato deve essere subito *risolto* calcolando sulla propria macchina un **output** mediante l'utilizzo di un programma scritto da chi sta svolgendo la gara (non può essere risolto *a mano* nemmeno parzialmente). Tale output deve essere conforme ai requisiti presenti nella descrizione della prova stessa: è importante notare che possono esistere output *diversi* tra loro ma ugualmente *validi* ai fini della prova che si sta risolvendo. Inoltre, eventuali stampe di debug che si fossero usate durante lo sviluppo del programma devono essere rimosse.

Fase 4: consegna dell'output e del codice sorgente

Si consegna (facendo l'upload tramite la pagina della prova) il **codice sorgente** usato per risolvere la prova assieme allo specifico **output** calcolato in risposta allo specifico input richiesto. Entrambi questi file sono necessari: se dovesse mancare uno, la piattaforma di gara restituirebbe un errore. Il codice sorgente sottoposto deve corrispondere al programma utilizzato per risolvere il problema: sottoporre un output assieme ad un sorgente che non produce tale output (ad esempio: un sorgente che non compila, oppure un sorgente che compila ma che quando eseguito sull'input produce un output diverso) può causare, durante la fase di controllo da parte del Comitato Olimpico, l'azzeramento del punteggio ottenuto. Il Comitato può, a sua discrezione, squalificare chi durante la gara adotta comportamenti non rispettosi delle regole.

Fase 5: ricezione del feedback

Il server di gara darà la possibilità di *annullare* o *confermare* la sottoposizione: in caso di conferma, l'input ad essa relativo non potrà essere riutilizzato in futuro. Una volta confermata la sottoposizione verrà mostrato il **feedback**, ovvero il *resoconto di quanti punti sono stati totalizzati* dalla sottoposizione. Sarà quindi possibile, se lo si desidera, richiedere un nuovo input (tornando alla **Fase 1**) per provare a migliorare il punteggio ottenuto.

Come lo scorso anno, ogni problema consisterà in 30 casi di test, suddivisi in 10 gruppi da 3 casi ciascuno. Ogni gruppo di 3 casi di test avrà un valore di **5 punti**, che si ottengono solo in caso il programma li abbia risolti correttamente **tutti e 3** (zero punti se anche uno solo dei 3 casi è sbagliato o mancante). Il punteggio assegnato ad una sottoposizione è quindi pari a **5*G**, dove **G** è il numero di gruppi di 3 casi di test che sono stati interamente risolti nella sottoposizione. Il punteggio assegnato ad un problema sarà quindi il **massimo** dei punteggi ottenuti sul problema tra tutte le sottoposizioni effettuate. Il punteggio totale della gara è la **somma** del punteggio di ogni problema. Lo stile di programmazione non ha alcun effetto sulla valutazione, così come il tempo usato per risolvere un input, purché l'esecuzione del programma sia abbastanza rapida da consentire la consegna dell'output entro la scadenza del timeout dell'input (ed entro il termine della gara).

6. Risultati fase territoriale e preparazione alla fase nazionale

Entro il 20 aprile 2026, i risultati degli studenti nella fase territoriale saranno pubblicati online. Verranno selezionati per proseguire con la formazione AlgoBadge i seguenti partecipanti:

- Gli studenti invitati in presenza ad almeno un corso di formazione OII svolto nell'anno 2025/2026 che rispettano i requisiti per partecipare alle IOI 2027, anche se non hanno effettuato la gara territoriale;
- Eventuali studenti che si sono particolarmente distinti alla fase finale dei Giochi di Fibonacci;
- Il miglior studente per punteggio di ogni sede territoriale;
- Le migliori 12 studentesse per punteggio a livello nazionale;
- I migliori 24 studenti e studentesse del biennio per punteggio a livello nazionale;
- Gli ulteriori studenti che hanno raggiunto un punteggio soglia definito dal comitato.

In caso di pari merito, saranno favoriti gli studenti provenienti da anni di corso inferiore e più giovani. Questi studenti dovranno raggiungere il **badge di bronzo** entro **domenica 24 maggio 2026**. Dopo quella data sarà fatto un controllo di regolarità e completamento, con possibilità di recupero delle irregolarità ed eventuali mancanze segnalate fino a **domenica 28 giugno**, dopo cui si farà un ultimo controllo. Tra gli studenti che supereranno questo controllo, saranno invitati a partecipare in presenza i seguenti partecipanti:

- Gli studenti invitati in presenza ad almeno un corso di formazione OII dell'anno 2025/2026;
- Il miglior studente per punteggio di ogni regione italiana;
- Le migliori 8 studentesse per punteggio a livello nazionale;
- I migliori 16 studenti e studentesse del biennio per punteggio a livello nazionale;
- Gli ulteriori studenti che hanno raggiunto un punteggio soglia definito dal comitato.

Entro martedì 30 giugno sarà quindi pubblicato l'elenco ufficiale dei partecipanti in presenza alla selezione nazionale. Gli altri studenti, insieme ad ulteriori studenti che si siano distinti in altre competizioni scientifiche nazionali o internazionali a discrezione del Comitato Olimpico, potranno continuare a lavorare sul percorso AlgoBadge fino a **domenica 6 settembre**. Coloro tra questi che raggiungeranno il badge di bronzo con regolarità a quella data potranno comunque partecipare alla selezione nazionale ma **in modalità online**. Gli studenti ammessi sia in presenza che online verranno invitati ad una sessione di prova detta **pre-OII** del sistema di gara delle nazionali, che si svolgerà nel mese di settembre con problemi originali.

7. Terza fase: selezione nazionale (24-26 settembre 2026)

Gli studenti ammessi verranno invitati a partecipare a un evento in presenza di selezione, formazione e networking, previsto da 24 al 26 settembre 2026. La gara si svolgerà nella giornata di **venerdì 25 settembre 2026**, indicativamente dalle ore **09:00** alle ore **14:00**, e consisterà nella risoluzione di problemi algoritmici tramite scrittura di programmi al computer, di formato analogo alle gare internazionali. Il giorno prima della gara verrà effettuata una prova di gara per verificare il corretto funzionamento del sistema e verranno fornite eventuali informazioni logistiche di dettaglio aggiuntive. La partecipazione è gratuita e sulle singole scuole graveranno solo le eventuali spese di viaggio. Nel solo caso di comprovate difficoltà logistiche o sanitarie, gli studenti potranno fare richiesta di passaggio dalla partecipazione in presenza ad una partecipazione online. Tutti i referenti territoriali saranno inoltre invitati a partecipare, a spese dell'organizzazione, con l'incarico di accompagnare eventuali studenti ammessi del loro territorio, se presenti. I referenti di sede distaccata potranno anche fare richiesta di partecipare, ma con le spese di trasporto e alloggio a carico della propria scuola. I partecipanti online saranno coinvolti nella sola sessione di prova e nella sessione di gara.

I problemi devono essere risolti tramite programmi scritti in **linguaggio C++** (con standard C++20 o successivo) che devono funzionare correttamente con qualsiasi input che rispetti i limiti assegnati nel testo. Il programma deve rispettare le modalità di input/output descritte nel testo del problema: nello sviluppo del programma è possibile utilizzare l'input/output da tastiera/video per eseguire debugging e testing ma questo dovrà essere rimosso dal sorgente prima di inviarlo al sistema di gara. I partecipanti possono scegliere quali

problemi risolvere e in quale ordine. I problemi riporteranno esempi, limiti di tempo e di uso di memoria massimo assegnati per la soluzione dei casi di test, e limiti e/o condizioni dei dati di ingresso.

I partecipanti, in presenza e online, saranno identificati attraverso un documento di riconoscimento in corso di validità. I partecipanti online dovranno rispettare un protocollo di sicurezza che comprenderà condivisione dello schermo e proctoring tramite videochiamata in tempo reale. Le postazioni di lavoro disponibili per lo svolgimento della prova non dovranno essere connesse ad Internet all'infuori del sistema di gara. È vietato consultare testi, manuali o appunti di qualsiasi genere, pena l'esclusione dalla prova. È vietato l'utilizzo di qualsiasi tipo di AI, con particolare riferimento a LLM in grado di scrivere codice. È vietato utilizzare qualsiasi dispositivo elettronico al di fuori del computer di gara. Ogni oggetto personale (tastiera, mouse, mascotte, viveri) che si voglia avere durante la gara deve essere portato durante la prova del giorno precedente per essere esaminato e approvato dallo staff. Tastiera e mouse non devono essere programmabili o wireless. È vietato tentare di interferire in qualunque modo con il regolare svolgimento della gara, ed eventuali violazioni o situazioni di copiatura rilevate dallo staff saranno **causa di squalifica**.

Nessun partecipante potrà lasciare la gara prima che siano trascorsi 90 minuti dall'inizio della prova, e negli ultimi 30 minuti prima della fine della prova, eccetto in caso di giustificate emergenze. I partecipanti potranno rivolgere domande sul testo dei problemi e richieste di supporto tecnico tramite il sistema di gara. Esclusivamente nel caso in cui un problema tecnico renda inaccessibile il sistema di gara, sarà possibile chiedere assistenza allo staff alzando la mano (se in presenza) o tramite la videochiamata (se online).

A ogni problema viene assegnato un punteggio massimo pari a 100. Ogni sottoposizione viene valutata su un certo numero di casi di prova suddivisi in subtask. Il punteggio di una sottoposizione per un subtask è determinato dai casi di prova che vengono risolti correttamente entro i limiti di tempo e memoria previsti dal problema, secondo le regole di assegnazione del punteggio indicate nella descrizione del problema stesso. Il punteggio di un partecipante per un subtask è pari al miglior punteggio che una sua sottoposizione ha ottenuto su quel subtask. Per ogni problema, il punteggio finale ottenuto da un partecipante è la somma dei punteggi per ogni subtask di quel problema. Stile di programmazione, numero di sottoposizioni e tempo di sottoposizione (se non per la determinazione dell'ultima sottoposizione) non hanno effetto sul punteggio.

8. Risultati fase nazionale e preparazione ai corsi di formazione

I primi classificati alle Olimpiadi Italiane di Informatica 2026 saranno premiati con:

- un dodicesimo dei partecipanti riceve una **MEDAGLIA D'ORO**
- un sesto dei partecipanti riceve una **MEDAGLIA D'ARGENTO**
- un quarto dei partecipanti riceve una **MEDAGLIA DI BRONZO**
- gli altri partecipanti che hanno raggiunto almeno 100 punti ricevono la **MENZIONE D'ONORE**

approssimando i numeri tenendo conto di eventuali ex aequo.

I vincitori delle medaglie d'oro, insieme ad alcuni vincitori di medaglie e menzioni che, a insindacabile giudizio del Comitato, presentino particolari e motivati meriti, saranno dichiarati Probabili Olimpici 2027 e saranno ammessi alla successiva fase di corsi di formazione in presenza, a patto di rispettare i requisiti richiesti per rappresentare l'Italia alle IOI 2027 (vedi [regolamento](#)). L'ammissione all'allenamento in presenza è condizionata al raggiungimento del **badge di diamante** sulla piattaforma di allenamento AlgoBadge, entro una data che sarà specificata contestualmente all'ammissione (e che sarà indicativamente qualche giorno prima della data dell'allenamento). Sarà inoltre possibile partecipare alla successiva fase di allenamento in modalità unicamente online anche senza essere dichiarati Probabili Olimpici per altri studenti che raggiungano il badge di diamante su AlgoBadge in tempo, con sottoposizioni originali e non copiate da altri studenti o da internet o scritte da AI, e rispettino i requisiti per rappresentare l'Italia alle IOI 2027. Similmente, per il 2025/2026, gli studenti interessati potranno fare richiesta di partecipazione ai corsi di formazione in modalità online al link:

<https://forms.gle/Wnhj2YL4TSwFx3tu8>

9. Quarta fase: corsi di formazione (anni scolastici 2026/2027 e 2025/2026)

Durante l'anno scolastico si terranno i corsi di formazione, suddivisi in più momenti dell'anno e durante i quali verranno effettuate ulteriori selezioni di natura eliminatoria, che porteranno alla formazione delle squadre olimpiche che verranno inviate alle IOI e ad altre competizioni internazionali. Le spese di viaggio e soggiorno per tutti i partecipanti in presenza saranno a carico dell'organizzazione. I partecipanti online che superino le selezioni eliminatorie verranno invitati in presenza per i successivi incontri di formazione.

Similmente, per essere ammissibile ai corsi di formazione che si svolgeranno nell'anno 2025/2026 uno studente dovrà aver completato il percorso AlgoBadge al livello *Diamante* entro l'8 novembre 2025 alle 23:59, e rispettare i requisiti richiesti per rappresentare l'Italia alle IOI 2026.

10. Ulteriori attività e iniziative

In alcune regioni, grazie a protocolli di intesa stipulati con le locali Università, si svolgeranno attività di formazione per gli studenti e di sostegno per i docenti con riferimento alle varie fasi di selezione. Attualmente sono attive collaborazioni con le università di Bologna, Campobasso, Catania, L'Aquila, Milano, Pisa, Roma, Torino e Trento. Altre potranno avviarsi nei prossimi mesi.

Eventuali chiarimenti possono essere richiesti telefonando al numero [02 7645 5025](tel:0276455025) (Segreteria delle Olimpiadi di Informatica), scrivendo all'indirizzo info@olimpiadi-informatica.it, oppure accedendo al sito ufficiale della manifestazione www.olimpiadi-informatica.it dove sono disponibili regolamenti e aggiornamenti, risorse sulle competizioni degli scorsi anni e collegamenti a siti esterni di riferimento.

Altre risorse web con materiali relativi alle Olimpiadi Italiane di Informatica sono raggiungibili a partire dalla pagina indice olinfo.it. In particolare da qui è possibile accedere al [forum](#), al percorso didattico [AlgoBadge](#), e alle piattaforme di allenamento per le gare [scolastiche](#), [territoriali](#) e [nazionali](#), che presentano raccolte di problemi di programmazione (catalogati in base alla difficoltà di soluzione) e consentono allo studente di sottoporre le proprie soluzioni per verificarne la correttezza.

La partecipazione all'iniziativa comporta l'accettazione implicita del presente regolamento.

Roma, 10 ottobre 2025

Comitato per le Olimpiadi Italiane di Informatica

Il Presidente

Luigi Laura



Appendice B

Pseudocodice

Pseudocodice (4.0)

Una guida completa alla **lettura e comprensione** degli
esercizi di programmazione delle selezioni scolastiche

Lo Staff delle OII

Ottobre 2022

Sullo pseudocodice nella selezione scolastica

Da anni, gli esercizi della selezione scolastica delle OII sono suddivisi in tre parti:

- esercizi di carattere logico-matematico;
- esercizi di programmazione;
- esercizi di carattere algoritmico.

A partire dalla selezione scolastica di novembre 2018 abbiamo introdotto un importante cambiamento relativo alla forma in cui gli **esercizi di programmazione** vengono proposti: invece di fornire del codice vero e proprio (in due dei linguaggi più “popolari” tra quelli insegnati in Italia a livello scolastico: C e Pascal) abbiamo cominciato a utilizzare uno **pseudocodice**. Con *pseudocodice* intendiamo un linguaggio che permette di descrivere programmi usando una sintassi naturale, “umana”, senza le rigide regole di un linguaggio di programmazione.

Lo stile di pseudocodice che abbiamo adottato è un’evoluzione di quello già in uso alle *Olimpiadi del Problem Solving*, migliorandolo con vari aiuti visuali e alcune sintassi più intuitive. Ricordiamo comunque che la sintassi dello pseudocodice non è da considerare esaustiva: è possibile che alcuni esercizi utilizzino dei costrutti non documentati in questa guida. Se questo accadrà, i “nuovi” costrutti saranno sempre spiegati nel testo dell’esercizio e pensati per essere facilmente comprensibili.

Attenzione! Questo cambiamento si riferisce **solo agli esercizi di programmazione** contenuti nella **selezione scolastica**. La fase territoriale e nazionale rimangono invariate e richiedono sempre la scrittura di programmi “veri”.

È importante notare che, sebbene le fasi successive alla scolastica facciano uso di linguaggi veri e propri, in questo caso esiste una fondamentale distinzione: ciò che valutiamo in questa gara infatti è l’abilità nel **leggere e capire** del codice già scritto, non quella di **scriverne**. Siamo convinti che lo pseudocodice sia particolarmente adatto allo scopo di questa gara.

Con lo pseudocodice, infatti, gli esercizi di programmazione diventano alla portata di tutti gli studenti, non necessariamente di quelli che sanno già programmare, o il cui curriculum scolastico comprende l’informatica: è sufficiente avere curiosità e voglia di imparare e mettersi in gioco!

Novità dell'edizione 2022

Quest'anno abbiamo rinnovato la guida, espandendola per una documentazione più comprensiva dello pseudocodice, con spiegazioni dettagliate e allo stesso tempo comprensibili. Inoltre, lo pseudocodice stesso ha subito delle modifiche:

- sono stati introdotti i cicli **for**;
- non esistono più le procedure, ma solo funzioni, che possono eventualmente non restituire nulla;
- il comando **output** è stato sostituito dalla funzione **output**;
- è stata introdotta la possibilità di “scambiare” il contenuto di due variabili;
- la formattazione dello pseudocodice è stata migliorata, usando anche dei colori per differenziare le varie componenti (parole chiave, variabili, etc.).

Guida rapida allo pseudocodice

Nella pagina successiva trovi un *cheat sheet*, una guida di riferimento sullo pseudocodice che riassume in una tabella i punti salienti della sintassi. Ogni riga della tabella contiene la sintassi di un elemento dello pseudocodice, una breve descrizione, ed eventualmente uno o più esempi del suo utilizzo. Questa guida rapida sarà anche fornita per consultazione insieme al testo della selezione scolastica.

Pseudocodice	Descrizione	Esempio
■ Variabili e tipi		
variable <i>i</i> : <i>integer</i>	Dichiarazione della variabile di tipo intero chiamata <i>i</i>	
variable <i>arr</i> : <i>integer</i> []	Dichiarazione di una variabile di tipo array di interi chiamata <i>arr</i>	
{var} ← {espr}	Assegnamento del valore dell'espressione {espr} alla variabile {var}	<i>i</i> ← 1 <i>a</i> ← 3 × <i>i</i> + 5 <i>arr</i> ← [3, 5/ <i>a</i> , 2]
<i>arr</i> [{espr}]	Variabile corrispondente all'elemento dell'array <i>arr</i> di indice {espr}	<i>a</i> ← <i>arr</i> [3] + 1 <i>arr</i> [<i>x</i> + 1] ← 2
(<i>a</i> , <i>b</i>) ← (<i>b</i> , <i>a</i>)	Scambio del valore delle variabili <i>a</i> e <i>b</i>	
■ Operatori		
+, −, ×, /, mod	Aritmetica: addizione, sottrazione (o negazione), moltiplicazione, divisione intera, resto della divisione intera (modulo)	<i>a</i> + <i>b</i> − <i>a</i> <i>i</i> ← <i>a mod</i> 10
==, ≠, <, ≤, >, ≥	Confronto: uguale, diverso, minore, minore o uguale, maggiore, maggiore o uguale	<i>a</i> == 7 2 × (<i>x</i> + 1) ≤ <i>y</i>
and , or , not	Operatori logici: e, o, non	<i>a</i> > <i>b</i> and <i>b</i> ≠ −1 not (<i>a</i> > 2 or <i>a</i> == 0)
■ Strutture di controllo		
if {condizione} then {corpo if} else {corpo else} end if	Struttura condizionale if ... else : se {condizione} è vera viene eseguito {corpo if} , altrimenti viene eseguito {corpo else} . La parte else può essere omessa	if <i>n mod</i> 2 == 0 then <i>i</i> ← 0 else <i>i</i> ← <i>n</i> − 1 end if
while {condizione} do {corpo} end while	Ciclo while : il blocco {corpo} viene ripetuto fintanto che {condizione} è vera	while <i>i</i> < <i>n</i> do <i>sum</i> ← <i>sum</i> + <i>i</i> <i>i</i> ← <i>i</i> + 7 end while
for {indice} in {intervallo} do {corpo} end for	Ciclo for : il blocco {corpo} viene eseguito mentre la variabile {indice} itera sui valori in {intervallo} , specificato come [<i>a</i> ... <i>b</i>], che significa "tutti i numeri da <i>a</i> (incluso) fino a <i>b</i> (escluso)"	for <i>i</i> in [0 ... <i>n</i>) do <i>arr</i> [<i>i</i>] ← −1 end for (assegna −1 a tutti gli elementi di un array <i>arr</i> di lunghezza <i>n</i>)
■ Funzioni		
function fun (<i>var1</i> : <i>tipo1</i> , <i>var2</i> : <i>tipo2</i> , ...) → <i>ritorno</i> {corpo} end function	Funzione con parametri <i>var1</i> , <i>var2</i> , etc. Il tipo di ritorno → <i>ritorno</i> può essere omesso. Il valore viene restituito tramite la parola chiave return	function add (<i>a</i> : <i>integer</i> , <i>b</i> : <i>integer</i>) → <i>integer</i> return <i>a</i> + <i>b</i> end function
fun () fun (<i>arg1</i> , <i>arg2</i> , ...)	Chiamata alla funzione fun (rispettivamente senza argomenti e con argomenti). La funzione output stampa il valore di una variabile oppure una stringa fissata. Le funzioni min e max restituiscono risp. il minimo e il massimo di due interi	return add (<i>a</i> , <i>b</i>) <i>m</i> ← very_big_integer () output (<i>x</i>) output ("string") min (<i>a</i> , <i>b</i>) max (<i>a</i> , <i>b</i>)

Documentazione dello pseudocodice

In questa sezione della guida definiamo la **sintassi** dello pseudocodice. Vale a dire, forniamo le nozioni necessarie per la lettura e comprensione di un programma scritto in pseudocodice. Le spiegazioni saranno abbastanza “formali” da essere precise, ma non tecniche, e saranno tutte corredate da piccoli esempi. Nella sezione successiva, invece, troverai molti altri esempi più completi, alcuni dei quali tratti da selezioni scolastiche passate. Dopo aver letto la documentazione, dovresti essere in grado di comprendere il funzionamento di questi programmi:

Programma 1	FizzBuzz	Programma 2	Uso degli array
1: function fizzbuzz(n : <i>integer</i>)		1: function maximum(n : <i>integer</i> , v : <i>integer</i> [])	
2: variable i : <i>integer</i>		→ <i>integer</i>	
3: i ← 1		2: variable m : <i>integer</i>	
4: while i ≤ n do		3: m ← v [0]	
5: if i mod 3 == 0 then		4: for i in [0 ... n) do	
6: if i mod 5 == 0 then		5: if v [i] > m then	
7: output("fizzbuzz")		6: m ← v [i]	
8: else		7: end if	
9: output("fizz")		8: end for	
10: end if		9: return m	
11: else		10: end function	
12: if i mod 5 == 0 then			
13: output("buzz")			
14: else			
15: output(i)			
16: end if			
17: end if			
18: i ← i + 1			
19: end while			
20: end function			

Componenti di un programma

Osservando i programmi 1 e 2 qui sopra, puoi notare che compaiono termini e simboli evidenziati in modo diverso: colori diversi, alcuni in grassetto e altri no, etc. Questi “blocchi fondamentali”, che esamineremo uno per uno, sono i seguenti:

- **parole chiave** (esempio: **while**, **end**, **variable**)
- **variabili** (esempio: **m**)
- **tipi** (esempio: **integer**, **integer[]**)
- **valori** (esempio: **1**, **42**)
- **operatori** (esempio: **>**, **mod**)

È anche evidente che alcune righe di codice sono spostate a destra (si dice che sono *indentate*). Una serie di righe consecutive con lo stesso *livello di indentazione* (cioè, precedute dallo stesso spazio bianco) si chiama **blocco**. I blocchi rappresentano parti di codice che vengono eseguite “consecutivamente” all’interno di una **funzione** o di una **struttura di controllo**. Ciascun blocco è prededuto da una riga che inizia con una parola chiave, ed è seguito da una riga che inizia con la parola chiave **end**.

Parole chiave

Le **parole chiave** sono parole che hanno un significato specifico all’interno di un programma, e, in quanto tali, non possono essere usate altrove (ad esempio, come nome di variabili — vedi sezione successiva). Sono formattate in **blu e grassetto**. Questa è la lista delle parole chiave:

- | | |
|-------------------|-------------------|
| • function | • for |
| • return | • in |
| • if | • while |
| • then | • do |
| • else | • end |
| | • variable |

La funzione di ciascuna parola chiave verrà esaminata quando parleremo del contesto in cui compare.

Variabili e tipi

Le **variabili** sono fondamentali in un programma. Nel nostro pseudocodice sono formattate in **azzurro e grassetto**.

Una variabile è una sorta di contenitore per un valore, che può essere un numero, o qualcosa di più complicato come una “sequenza di numeri”. Queste “tipologie” di variabile si chiamano, appunto, **tipi**, e sono formattati in **blu e corsivo**. Esiste un solo tipo semplice contemplato dal nostro pseudocodice:

- **integer**: è il tipo che rappresenta numeri interi, ovvero $\{\dots, -2, -1, 0, 1, 2, \dots\}$.

Esiste una categoria di tipi “composti”, gli **array**, che sono trattati alla fine di questa sezione. **Attenzione!** Alcuni degli esempi che seguono usano gli array, quindi può essere una buona idea riprenderli dopo aver letto il paragrafo su di essi.

Una variabile è identificata da un nome: a variabili diverse corrispondono nomi diversi. Un programma usa le variabili per manipolare valori che possono cambiare durante l'esecuzione del codice. Le variabili possono essere:

- *dichiarate*. Dichiarare una variabile è obbligatorio, e vuol dire comunicare al programma che tale variabile esiste. Per questo, va fatto prima di qualunque altra operazione che coinvolge la variabile. Una dichiarazione di variabile inizia con la parola chiave **variable**, seguita dal nome della variabile, dai due punti, e dal tipo della variabile.
Attenzione! Appena dopo la dichiarazione, la variabile non contiene nessun valore: deve essere *inizializzata* assegnandone uno.

Questi sono esempi di dichiarazione:

variable i: integer
variable arr: integer[]

Queste **non** sono dichiarazioni:

variable i
arr: integer[]
integer a

- *inizializzate e modificate tramite assegnamento*. L'assegnamento è infatti l'operazione che assegna un nuovo valore alla variabile. Un assegnamento è costituito dal nome della variabile, seguito dall'operatore $\leftarrow$ (tratteremo gli operatori più avanti), seguito dal valore da assegnare. Quest'ultimo può essere un valore semplice, un'altra variabile, o più in generale un'**espressione** (anche di espressioni parleremo in seguito).

Questi sono esempi di assegnamenti validi:

```
i ← 1
a ← 3 × i + 5
arr ← [3, 5/a, 2]
```

Questo **non** è un assegnamento:

```
i == 1
```

Questo **non** è codice valido:

```
1 ← 3
```

- *usate nelle espressioni*. Per esempio, una variabile può essere sommata a un'altra variabile, o a un valore, o un'altra espressione, tramite l'operatore $+$. Oppure può essere confrontata, tramite operatori quali $==$, $<$, $\geq$, e altri. Tutto ciò verrà trattato meglio quando parleremo di operatori e di espressioni.

Array

Un array è una sequenza di valori tutti dello stesso tipo, che può avere lunghezza arbitraria (eventualmente lunghezza 0, e in tal caso si parla di *array vuoto*). Il tipo corrispondente si indica con il tipo base, seguito da parentesi quadre: `integer[]` è l'unico tipo array in questo pseudocodice, poiché `integer` è l'unico tipo base.

Il contenuto dell'array si indica con una lista di valori (o espressioni) inframezzati da virgole e racchiusi da parentesi quadre: `[3, i, 1 - 2]`, `[100]`, `[]` (array vuoto).

Attenzione! Un array di lunghezza n è indicizzato, cioè numerato, da 0 a $n - 1$. L'indice i corrisponde all'elemento in posizione i . Per esempio, nell'array `[3, -1, 8]`, gli indici sono 0, 1 e 2, e l'indice 1 corrisponde al valore `-1`.

Gli elementi che costituiscono un array sono a loro volta variabili che possono essere usate e assegnate. Per accedere ad un elemento di un array, si scrive il nome dell'array, seguito dall'indice racchiuso da parentesi quadre: `arr[3]`, `arr[i + 1]`.

Nota che l'indice può essere un numero ma anche un'espressione (contenente anche delle variabili). Una volta acceduto ad un elemento, lo si può usare in espressioni e gli si possono assegnare valori: `arr[0] ← 0`, `arr[i] ← arr[i] + 1`.

Approfondimento: il caso delle stringhe. Forse hai notato che, nel programma 1, compaiono delle parole racchiuse da apici doppi (virgolette). Ne

incontrerai ancora in seguito. Si tratta di quelle che in informatica vengono chiamate *stringhe*, cioè sequenze di caratteri. Nella maggior parte dei linguaggi di programmazione esiste un tipo associato alle stringhe, ma nel nostro pseudocodice non esiste un tipo stringa: le stringhe esistono soltanto come possibili argomenti della funzione **output**, esattamente come nel programma FizzBuzz.

Valori

Un **valore** è un'entità fissa, costante, che compare nel codice in quanto tale: un valore non può essere modificato. Ogni valore deve appartenere a uno dei tipi descritti in precedenza. Esistono quindi valori interi, di tipo *integer*, e valori di tipo array *integer[]*. Nel nostro pseudocodice i valori sono formattati in **giallo**.

Questi sono esempi di uso dei valori:

a $\leftarrow$ 0
1 + 7

Questa **non** è un'operazione valida:

[] $\leftarrow$ [3, -1, 8]

Attenzione! In questa guida, a volte usiamo la parola *valore* per riferirci al risultato di un'espressione (per esempio, 3×5 risulta nel valore 15), che è un'altra cosa (non è un'entità "scritta" nel codice). Quindi fai attenzione, il significato della parola deve essere talvolta dedotto dal contesto.

Operatori

Un **operatore** è un simbolo, o una parola, che combina i risultati di due espressioni (valori), producendo un nuovo risultato (valore). Gli operatori sono formattati in **viola e grassetto** (o solo viola per i simboli).

Esistono varie classi di operatori:

- *operatori aritmetici*: +, −, ×, /, **mod** (addizione, sottrazione, moltiplicazione, divisione intera, resto della divisione intera). Questi prendono due interi

e restituiscono un intero. Il significato dei primi tre dovrebbe essere chiaro. L'operatore `/` di divisione intera produce come risultato la *parte intera* (cioè, approssimazione verso lo 0) del quoziente di due interi: ad esempio, `7 / 3` restituisce 2 (perché $7/3 = 2.333\dots$). L'operatore modulo `mod` produce come risultato il resto della divisione di due interi: ad esempio, `7 mod 3` restituisce 1. Infine, l'operatore `-` può essere anche anteposto ad un'espressione, cambiandone il segno (esempio: se il valore di `a` è 3, il valore di `-a` è -3).

- *operatori di confronto*: `==`, `≠`, `<`, `≤`, `>`, `≥` (uguale, diverso, minore, minore o uguale, maggiore, maggiore o uguale). Come dice il nome, confrontano due interi. Ad esempio, `1 == 3 + (-2)` è vera, mentre `7 < 7` è falsa.

Attenzione! Il simbolo di uguaglianza è proprio quello che vedi: due uguali “normali” uno dopo l'altro, separati da uno spazio. L'uguale singolo non viene usato in questo pseudocodice, per evitare confusioni con l'assegnamento.

- *operatori logici*: `and`, `or`, `not`. I primi due operatori combinano due espressioni (termini) contenenti operatori di confronto, o altri operatori logici. `and` restituisce vero se entrambi i termini sono veri, e falso altrimenti; `or` restituisce vero se almeno uno dei termini è vero, e falso altrimenti. `not` è un cosiddetto operatore *unario*, perché agisce su una sola espressione. Restituisce vero se l'espressione è falsa, e falso se l'espressione è vera. Per esempio: `(a == b) and (a ≠ b)` è falsa qualunque siano i valori di `a` e `b` (perché?), mentre `not (7 ≥ 9)` è vera.

Approfondimento: valori di verità. Nella descrizione degli operatori di confronto e logici, abbiamo parlato di “vero” e “falso”. Questi sono detti *valori di verità*, e presuppongono l'esistenza di un tipo opportuno, presente in tutti i linguaggi veri. Il nostro pseudocodice non contempla l'esistenza esplicita di questo tipo (proprio come per le stringhe): i valori di verità possono essere usati solo nei punti decisionali delle strutture di controllo (che vedremo più avanti).

La precedenza tra gli operatori aritmetici è quella tradizionale: `×` e `/` hanno precedenza maggiore di `+` e `-`. Per quanto riguarda `mod`, questo ha la stessa precedenza di moltiplicazione e divisione, perciò useremo sempre le parentesi tonde per non creare ambiguità. Gli operatori di confronto hanno meno precedenza di quelli aritmetici, ma più precedenza di quelli logici. Anche qui, comunque, useremo parentesi dove necessario per prevenire ambiguità.

Espressioni

Un'espressione è un pezzo di codice dotato di un proprio valore, cioè che può essere *valutato*. Ad esempio, sia i valori sia le variabili di cui abbiamo parlato prima sono semplici espressioni. Ma espressioni sono anche cose più complesse, come $(i + j \times 3) \bmod 10$.

Le espressioni sono importanti perché possono essere assegnate a una variabile, passate come argomento a una funzione (vedi più avanti), ed essere usate a loro volta in un'espressione. Espressioni sono:

- singole variabili (inclusi gli elementi di un array);
- singoli valori (inclusi quelli di tipo array);
- una chiamata a funzione (che vedremo più avanti);
- la composizione di due espressioni tramite un operatore: ad esempio, se `espr1` ed `espr2` sono espressioni, anche `espr1 + espr2` è un'espressione;
- $\neg(\text{espr})$, dove `espr` è un'espressione che restituisce un intero, e `not (espr)`, dove `espr` è un'espressione che restituisce vero o falso.

Per verificare se hai capito, puoi provare a individuare **tutte** le espressioni contenute nella seguente istruzione: `a ← a - b × 2` (suggerimento: sono 6).

Strutture di controllo

Un programma composto da una semplice successione di istruzioni limita molto le possibilità. Considera il seguente pseudocodice:

Programma 3	Sequenza di istruzioni
--------------------	------------------------

```
1: variable a: integer
2: variable b: integer
3: variable c: integer
4: a ← 1
5: b ← 2
6: c ← 3
7: variable sum: integer
8: sum ← a + b + c
9: output(sum)
```

In questo frammento di pseudocodice, ogni riga è una singola istruzione. L'istruzione `output(sum)`, come vedremo tra poco, serve a stampare in output il valore di `sum`. Quello che il programma fa è sommare tre numeri “fissati” nel programma (1, 2 e 3): sostanzialmente qualcosa che avremmo potuto fare a mano. È in questo senso che è necessario introdurre complessità allo pseudocodice, poiché altrimenti non c'è molto che si possa fare. Le strutture di controllo hanno questo scopo (insieme alle funzioni, che vedremo nella sezione successiva).

Una **struttura di controllo** è un costrutto che racchiude un blocco di pseudocodice, detto *corpo*. A seconda del tipo di struttura di controllo, le istruzioni contenute nel corpo possono essere eseguite solo sotto particolari condizioni, oppure ripetute più volte. Le strutture di controllo possono essere annidate, ovvero il corpo di una struttura può contenerne delle altre. Il nostro pseudocodice utilizza tre tipi di strutture di controllo:

- Le strutture condizionali `if` e `if ... else`, con la seguente sintassi:

```
if {condizione} then
    {corpo if}
end if
```

```
if {condizione} then
    {corpo if}
else
    {corpo else}
end if
```

Qui, `{condizione}` è un'espressione che restituisce vero o falso (ad esempio un confronto). Se è vera, viene eseguito `{corpo if}`. Per `if ... else`, qualora il valore di `{condizione}` sia falso, viene eseguito invece `{corpo else}`.

Per esempio, questo programma assegna alla variabile `b` l'intero 100 se `a` vale 0, e 101 se `a` vale 1 (negli altri casi non succede niente):

Programma 4 Uso di `if` e `if ... else`

```
1: variable b: integer
2: if a == 0 then
3:     b ← 100
4: else
5:     if a == 1 then
6:         b ← 101
7:     end if
8: end if
```

- Il ciclo **while**, con la seguente sintassi:

```
while {condizione} do
  {corpo}
end while
```

In questo caso, **{corpo}** viene ripetuto fintanto che **{condizione}** è vera. Quando il programma incontra un **while**, controlla prima se la condizione è vera: se non lo è, salta completamente il blocco e continua l'esecuzione. Se lo è, esegue una volta il corpo, e poi controlla nuovamente se la condizione è vera, ri-eseguendo il corpo in tal caso. Questo si ripete finché la condizione diventa falsa. Nota che questo ha senso se il valore di **{condizione}** dipende da quello che accade dentro **{corpo}**; altrimenti, è possibile che **{condizione}** sia sempre vera e il programma non esca mai dal ciclo: si parla in questo caso di *ciclo infinito*. Ad esempio, questo programma calcola la somma dei multipli di 7 minori di n :

Programma 5 Uso di **while**

```
1: variable i: int
2: variable sum: int
3: i ← 0
4: sum ← 0
5: while i < n do
6:   sum ← sum + i
7:   i ← i + 7
8: end while
```

- Il ciclo **for**, con la seguente sintassi:

```
for {indice} in {intervallo} do
  {corpo}
end for
```

Vediamo cosa sono **{indice}** e **{intervallo}**. Partiamo dal secondo, che, come dice il nome, è un intervallo (di numeri interi), ovvero un insieme di numeri consecutivi. Per esempio, $\{-1, 0, 1, 2, 3\}$ e $\{7\}$ sono intervalli, mentre $\{3, 5\}$

non lo è. Dato che scrivere esplicitamente tutti i numeri di un intervallo è scomodo (e a volte impossibile), nello pseudocodice usiamo una notazione speciale per gli intervalli, ispirata da una tradizione matematica: $[a \dots b)$ indica l'intervallo di numeri che inizia da a e finisce in $b - 1$ (incluso). Ovvero, l'estremo destro dell'intervallo è escluso. Quindi $[1 \dots 3)$ è l'intervallo $\{1, 2\}$ (si parla di “intervallo semiaperto”). Questa notazione può confondere all'inizio, ma il motivo di avere intervalli semiaperti è che rende più naturale iterare sugli array, ed è una prassi comune in molti linguaggi di programmazione.

Per quanto riguarda **{indice}**, esso è il nome di una nuova variabile temporanea che “scorre” sugli elementi dell'intervallo dal più piccolo al più grande. Chiamiamola **i** (spesso i nomi utilizzati per gli iteratori sono **i**, **j**, **k**, ...). È temporanea nel senso che esiste solo all'interno del **for**, e sparisce non appena il programma esce dal ciclo. Non va dichiarata (basta scriverne il nome), ed è sottinteso che sia di tipo *integer*. Il corpo del **for** viene eseguito un numero di volte pari alla lunghezza dell'intervallo (il numero di elementi che contiene). Durante la prima iterazione, **i** assume il valore a (l'estremo sinistro dell'intervallo). Durante la seconda iterazione, assume il valore $a + 1$. E così via, fino all'ultima iterazione, durante la quale assume il valore $b - 1$. La variabile **i** non viene mai modificata all'interno del corpo del ciclo.

Vediamo alcuni esempi:

Programma 6 for semplice

```
1: for i in [0 ... n) do
2:   output(i)
3: end for
```

Programma 7 Somma di array

```
1: variable sum: integer
2: sum ← 0
3: for i in [0 ... n) do
4:   sum ← sum + v[i]
5: end for
```

Programma 8 Ripetizione

```
1: variable equal_pair: integer
2: equal_pair ← 0
3: for i in [0 ... n) do
4:   for j in [i + 1 ... n) do
5:     if v[i] == v[j] then
6:       equal_pair ← 1
7:     end if
8:   end for
9: end for
```

Il programma 6 stampa in output i numeri da 0 a $n - 1$. Il programma 7 calcola la somma degli elementi di un array **v** di lunghezza **n**. L'ultimo programma, 8, è un po' più complesso. Contiene due cicli **for** annidati. Il primo itera (tramite **i**) sugli elementi dell'array **v**, il secondo itera sugli elementi dell'array **che vengono dopo i**. Questo è il modo standard di "scandire" tutte le coppie di indici distinti. Quindi, alla fine dell'esecuzione, **equal_pair** vale **1** se e solo se è stata incontrata una coppia di elementi uguali, cioè una ripetizione.

Funzioni

Nei paragrafi precedenti abbiamo fatto spesso riferimento alle funzioni, senza mai chiarire cosa fossero. È finalmente arrivato il momento.

Una **funzione** è un blocco di pseudocodice, anche in questo caso chiamato *corpo*, racchiuso dalle parole chiave **function** e **end function**. Ci sono due motivazioni principali per l'utilizzo delle funzioni:

- permettere di riutilizzare parti di pseudocodice nel programma (a differenza dei cicli **while** e **for**, dove il corpo viene eseguito per più volte consecutive, una funzione può essere invocata, o chiamata, in punti arbitrari del programma);
- rendere possibile la **ricorsione**, ovvero la capacità di una funzione di invocare se stessa.

Le componenti di una funzione sono:

- il **nome**: È ciò che identifica la funzione, e grazie al quale la si può invocare. Nel nostro pseudocodice, i nomi di funzioni sono formattati in **blu**.
- il **corpo**, di cui abbiamo già parlato: È la sequenza di istruzioni che viene eseguita quando la funzione viene chiamata.
- la lista di **parametri**: È una sequenza della forma **var1: tipo1**, **var2: tipo2**, ... che indica quanti valori, e di quali tipi, vanno "passati" come **argomenti** alla funzione quando questa viene chiamata. Una funzione può non avere parametri.
- il tipo del **valore di ritorno** (opzionale): Una funzione non può soltanto "fare" qualcosa, ma può anche *restituire* un valore — ad esempio, una funzione che somma due interi restituisce un intero. In questo caso, il tipo del valore di ritorno va indicato.

La sintassi di una funzione è la seguente (in alto senza valore di ritorno, in basso con valore di ritorno):

```
function fun(var1: tipo1, var2: tipo2, ...)  
  {corpo}  
end function
```

```
function fun(var1: tipo1, var2: tipo2, ...) → ritorno  
  {corpo}  
end function
```

Il nome della funzione è **fun**, tra parentesi tonde ci sono gli eventuali parametri (se non ce ne sono, si scrive semplicemente **fun()**), e *ritorno* è il tipo del valore di ritorno. Le entità **var1**, **var2**, ... sono variabili che possono essere usate solo dalla funzione, non dal codice esterno. I loro valori sono definiti solo al momento della chiamata a funzione, e vengono passati come argomenti (vedi sotto nel paragrafo dedicato). In particolare, possono essere diversi in chiamate diverse.

La parola chiave **return**

All'interno del corpo, è possibile usare la parola chiave **return** per restituire un valore (solo per le funzioni con valore di ritorno). Quando viene incontrato un **return**, l'esecuzione della funzione si interrompe e riprende dal punto in cui era stata chiamata; alla chiamata di funzione viene sostituito il valore che ha restituito.

Ad esempio, considera il seguente programma:

Programma 9 Funzioni e valore di ritorno

```
1: function add(a: integer, b: integer) → integer  
2:   return a + b  
3: end function  
4: variable x: integer  
5: variable y: integer  
6: x ← -2  
7: y ← 5  
8: variable sum: integer  
9: sum ← add(x, y)
```

La funzione `add` prende due parametri interi, `a` e `b`, e restituisce un intero. L'intero restituito corrisponde alla somma dei due parametri, come specificato dal `return` seguito dall'espressione `a + b`. Alla riga 10, la funzione viene chiamata dal programma, passando come argomenti le variabili `x` (che vale -2) e `y` (che vale 5). Pertanto, `add` restituisce $-2 + 5 = 3$, e questo valore viene assegnato a `sum`.

La parola chiave `return` può essere usata più di una volta nella stessa funzione. Per esempio, supponiamo di voler scrivere una funzione che restituisca il valore assoluto di un intero n (cioè, n stesso se $n \geq 0$, altrimenti $-n$). Potremmo farlo così:

Programma 10 Più di un `return`

```
1: function absolute_value(n: integer) → integer
2:   if n ≥ 0 then
3:     return n
4:   end if
5:   return -n
6: end function
```

Se ad `absolute_value` viene passato un valore ≥ 0 , l'esecuzione del programma entra nel corpo dell'`if` e viene restituito `n`. In questo caso, l'esecuzione si interrompe: il programma “esce” immediatamente dalla funzione. In caso contrario, l'`if` viene saltato e viene eseguito il secondo `return`.

Ci si aspetterebbe che le funzioni senza tipo di ritorno non abbiano bisogno di alcun `return`. In effetti è così, ma a volte è comodo poter interrompere l'esecuzione di una funzione senza usare costrutti condizionali (che appesantiscono il codice). Per questo, si può usare un `return` “vuoto” (cioè non seguito da nulla) in un punto qualsiasi della funzione. Per esempio, questa funzione fa la stessa cosa di `absolute_value` nel programma 10, ma stampa il valore assoluto in output anziché restituirlo:

Programma 11 `return` vuoto

```
1: function print_absolute_value(n: integer)
2:   if n ≥ 0 then
3:     output(n)
4:     return
5:   end if
6:   output(-n)
7: end function
```

Chiamare una funzione

Se non fosse possibile invocare, o più comunemente “chiamare”, una funzione, esse sarebbero inutili. Chiamare una funzione vuol dire spostare, temporaneamente, l’esecuzione del programma all’inizio della funzione stessa, “passando” una lista di argomenti, cioè i valori che assumono i parametri della funzione in quella chiamata. È importante chiarire la distinzione tra parametri e argomenti:

- i **parametri** sono le **variabili** che compaiono nella definizione della funzione, e, come tali, hanno un nome e un tipo;
- gli **argomenti** sono i **valori** (risultati di espressioni) che vengono passati alla funzione in una particolare chiamata.

Ovviamente, gli argomenti devono essere tanti quanti i parametri, e i loro tipi devono corrispondere. Abbiamo già visto un esempio di chiamata di funzione nel programma 9, ma anche tutte le volte in cui viene invocata **output**: questa, infatti, è una funzione senza tipo di ritorno con un unico parametro.

Funzioni ricorsive

La **ricorsione** è uno strumento molto potente nella programmazione, che non sarebbe possibile senza funzioni. Consiste nella possibilità di una funzione di chiamare se stessa, all’interno del proprio corpo. Quando ciò avviene, il programma ricomincia l’esecuzione della funzione con **nuovi argomenti** (quelli passati nella chiamata). Si tratta di un’esecuzione distinta da quella che l’ha invocata: quest’ultima resta “sospesa” finché la chiamata interna non termina, e poi riprende normalmente. I valori di tutte le variabili non vengono modificati. Una funzione che chiama se stessa almeno una volta si dice **funzione ricorsiva**.

Un esempio classico è il calcolo del fattoriale. Il fattoriale di un intero positivo n , indicato con $n!$, è il prodotto dei numeri tra 1 ed n , ovvero $1 \cdot 2 \cdots (n - 1) \cdot n$. Si può calcolare facilmente usando le strutture di controllo (come un ciclo **for**), ma è istruttivo vedere come farlo tramite una funzione ricorsiva:

Programma 12 Funzione ricorsiva

```
1: function factorial(n: integer) → integer
2:   if n == 1 then
3:     return 1
4:   end if
5:   return n × factorial(n − 1)
6: end function
```

La funzione sfrutta il fatto che $n! = n \cdot (n - 1)!$, come si vede alla riga 5. L'`if` alle righe 2-4 serve a fare in modo che la ricorsione si arresti: prima o poi, infatti, il valore di n diventerà 1, e a quel punto viene restituito 1 (il fattoriale di 1) senza ulteriori chiamate a `factorial`.

Funzioni di libreria

Il nostro pseudocodice mette a disposizione tre funzioni *di libreria*, ovvero disponibili senza dover essere definite: `min`, `max` e `output`. La funzione `min` prende come parametri due interi, e restituisce il minimo fra di essi. Analogamente, la funzione `max` prende come parametri due interi, e restituisce il massimo fra di essi. Ad esempio, chiamando `min(-3, 2)` viene restituito -3 , e chiamando `max(a, a + 1)` viene restituito in ogni caso il valore di $a + 1$. La funzione `output` può essere chiamata con un argomento di tipo intero per stamparlo in output, oppure può essere chiamata con la forma `output("pippo")` che stampa la parola *pippo* in output.

Scambio di variabili

Un'operazione spesso utile in programmazione è scambiare i valori di due variabili. Come farlo è meno ovvio di quanto sembri: per scambiare i valori di `a` e `b`, non possiamo semplicemente scrivere in sequenza gli assegnamenti `a ← b` e `b ← a`. Infatti, subito dopo il primo assegnamento entrambe le variabili avranno lo stesso valore di `b`: il valore di `a` è andato perso! Un modo per ovviare è introdurre una terza variabile “ausiliaria”, chiamiamola ad esempio `x`, in questo modo:

```
x ← a
a ← b
b ← x
```

In pratica, **x** serve a memorizzare il valore di **a** prima che venga sovrascritto da quello di **b**, così che possa poi essere assegnato a **b** stesso.

Dato che, come già detto, lo scambio è molto frequente, nello pseudocodice usiamo una espressione dedicata esclusivamente a questo scopo.

$$(a, b) \leftarrow (b, a)$$

L'espressione **(a, b)** può essere usata **esclusivamente** in questo contesto. Le variabili che compaiono a destra devono essere le stesse che compaiono a sinistra, ma in ordine invertito. Questa istruzione compie esattamente lo scambio delle due variabili: è come se fosse un “assegnamento simultaneo”.

Esempi di pseudocodice

Vediamo ora altri esempi concreti, che valgono più di mille parole.

Implementazioni di algoritmi classici

In questa sezione trovi una lista di brevi programmi in pseudocodice che implementano algoritmi più o meno semplici e di natura didattica. Sono tutti proposti sotto forma di funzione. Sono presenti descrizioni molto minimali: una spiegazione esauritiva dello pseudocodice è stata volutamente omessa, per incentivarti a comprendere autonomamente il funzionamento del programma.

Capire se un numero è primo

La funzione `is_prime` restituisce `1` se `n` è un numero primo, e `0` altrimenti.

Programma 13 Test di primalità

```
1: function is_prime(n: integer) → integer
2:   variable i: integer
3:   i ← 2
4:   while i × i ≤ n do
5:     if n mod i == 0 then
6:       return 0
7:     end if
8:     i ← i + 1
9:   end while
10:  return 1
11: end function
```

Stampare tutti i primi fino a 1000

La funzione `print_primes` si avvale della funzione `is_prime` dell'esempio precedente per stampare, in ordine, tutti i numeri primi compresi tra 1 e 1000.

Programma 14	Stampa i primi ≤ 1000
--------------	----------------------------

```
1: function print_primes()
2:   for i in [1 ... 1001) do
3:     if is_prime(i) == 1 then
4:       output(i)
5:     end if
6:   end for
7: end function
```

Ribaltare un array

La funzione `reverse` inverte gli elementi di un array `v` di lunghezza `n`.

Programma 15	Ribalta un array
--------------	------------------

```
1: function reverse(n: integer, v: integer[]) → integer[]
2:   for i in [0 ... n / 2) do
3:     variable j: integer
4:     j ← n - 1 - i
5:     (v[i], v[j]) ← (v[j], v[i])
6:   end for
7:   return v
8: end function
```

Massima somma di un prefisso

Un *prefisso* di un array è una sottosequenza contigua di elementi che parte da quello di indice 0. La funzione `max_prefix` trova la massima somma di un prefisso di un array `v` di lunghezza `n`.

Programma 16 Prefisso di somma massima

```
1: function max_prefix(n: integer, v: integer[]) → integer
2:   variable maximum: integer
3:   variable sum: integer
4:   maximum ← v[0]
5:   sum ← 0
6:   for i in [0 ... n) do
7:     sum ← sum + v[i]
8:     if sum > maximum then
9:       maximum ← sum
10:    end if
11:  end for
12:  return maximum
13: end function
```

Contare il numero di somme uguali a un numero dato

La funzione ricorsiva `count_sums` restituisce il numero di modi di scrivere un intero positivo n come somma **ordinata** di addendi positivi. Per esempio, $n = 4$ si scrive in 8 modi: $1 + 1 + 1 + 1$, $1 + 1 + 2$, $1 + 2 + 1$, $2 + 1 + 1$, $2 + 2$, $1 + 3$, $3 + 1$, 4 (si considera anche la somma composta dal solo addendo n).

Programma 17 Numero di somme uguali a n

```
1: function count_sums(n: integer) → integer
2:   if n == 1 then
3:     return 1
4:   end if
5:   variable answer: integer
6:   answer ← 1
7:   for i in [1 ... n) do
8:     answer ← answer + count_sums(i)
9:   end for
10:  return answer
11: end function
```

Somma di una sottosequenza

Questo programma è più complesso dei precedenti.

Una *sottosequenza* di un array è un sottoinsieme dei suoi elementi (anche non consecutivi). Dati un array di interi v di lunghezza n e un intero x , la chiamata `subsequence_sum(n, v, x, 0)` ritorna `1` se esiste una sottosequenza di v con somma x , e `0` altrimenti. Per convenzione, la sottosequenza vuota ha somma `0`.

Programma 18 Esiste una sottosequenza di somma x ?

```
1: function subsequence_sum(n: integer, v: integer[], x: integer, sum: integer) → integer
2:   if n == 0 then
3:     if sum == x then
4:       return 1
5:     end if
6:     return 0
7:   end if
8:   if subsequence_sum(n - 1, v, x, sum) == 1 then
9:     return 1
10:  end if
11:  if subsequence_sum(n - 1, v, x, sum + v[n - 1]) == 1 then
12:    return 1
13:  end if
14:  return 0
15: end function
```

Esempi da prove passate

Seguono degli esercizi tratti dalle ultime due edizioni della selezione scolastica. In fondo trovi le risposte a tutti gli esercizi, con una breve spiegazione per ciascuna di esse.

Selezioni scolastiche 2018-19, Esercizio 6

Calcolare per quante volte viene eseguito il ciclo `while` nel seguente pseudocodice:

Programma 19 2018-19, Esercizio 6

```
1: variable conta: integer
2: variable alfa: integer
3: variable beta: integer
4: conta ← 0
5: alfa ← 0
6: beta ← 0
7: while conta < 29 do
8:   if conta mod 3 == 1 then
9:     alfa ← alfa + 2
10:  else
11:    beta ← beta + 1
12:  end if
13:  conta ← conta + 2
14: end while
```

Selezioni scolastiche 2018-19, Esercizio 9

Sono date le seguenti funzioni:

Programma 20 2018-19, Esercizio 9

```
1: function mystery(a: integer, b: integer) → integer
2:   if 2 × a > b then
3:     return b
4:   else
5:     return a
6:   end if
7: end function
8:
9: function secret(a: integer, b: integer) → integer
10:  if a + b > mystery(a, b) then
11:    return a
12:  else
13:    return mystery(a, b)
14:  end if
15: end function
```

Quale valore viene restituito dalla chiamata `secret(24, 3)`?

Selezioni scolastiche 2020-21, Esercizio 6

Nel seguente programma, qual è il valore dell'ultimo intero che viene stampato durante l'esecuzione?

Programma 21	2020-21, Esercizio 6
---------------------	----------------------

```
1: variable v: integer[]
2: variable w: integer[]
3: v ← [4, 2, 6, 3, 5, 8, 9, 0, 7, 1]
4: w ← [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
5: for i in [0 ... 10) do
6:   w[v[i]] ← i
7: end for
8: for i in [0 ... 10) do
9:   output(w[i])
10: end for
```

Selezioni scolastiche 2020-21, Esercizio 7

È data la seguente funzione:

Programma 22	2020-21, Esercizio 7
---------------------	----------------------

```
1: function f(x: integer) → integer
2:   variable i: integer
3:   while x > 0 do
4:     if x mod 2 == 0 then
5:       x ← x / 2
6:     else
7:       x ← x - 1
8:     end if
9:     i ← i + 1
10:  end while
11:  return i
12: end function
```

Qual è il valore minimo da passare a x affinché f ritorni 5?

Selezioni scolastiche 2021-22, Esercizio 6

È data la seguente funzione:

Programma 23	2021-22, Esercizio 6
---------------------	----------------------

```
1: function f( $n$ : integer) → integer
2:   if  $n < 10$  then
3:     return  $f(n + 1) + 3$ 
4:   else
5:     if  $n == 10$  then
6:       return 7
7:     else
8:       return  $f(n - 2) - 1$ 
9:     end if
10:  end if
11: end function
```

Quanto vale $f(13)$?

Soluzioni

Selezioni 2018-19, Esercizio 6. La risposta è 15.

Tutto quello che succede nel corpo del `while` è irrilevante, perché non influenza il valore di `conta`! Quest'ultimo aumenta di 2 a ogni iterazione del ciclo. All'inizio vale 0, e il programma esce dal ciclo quando raggiunge il valore 30. Quindi il `while` viene eseguito $30/2 = 15$ volte.

Selezioni 2018-19, Esercizio 9. La risposta è 24.

Si tratta di simulare il programma: la chiamata `mystery(a, b)`, con `a` e `b` che valgono, rispettivamente, 24 e 3, restituisce `b`, cioè 3, perché $2 \cdot 24 > 3$. Allora la condizione `a + b > mystery(a, b)` è vera ($24 + 3 > 3$), per cui viene `mystery` restituisce il valore di `a`, ovvero 24.

Selezioni 2020-21, Esercizio 6. La risposta è 6.

L'ultimo intero a essere stampato è `w[9]`. Quindi, nel `for` alle righe 5-7 importa solo l'iterazione in cui `v[i]` assume il valore 9. Questo succede quando `i` vale 6, e l'istruzione `w[v[i]] ← i` assegna tale valore a `w[9]`.

Selezioni 2020-21, Esercizio 7. La risposta è 7.

Andiamo a ritroso. Il `while` si arresta quando `x` vale 0. All'iterazione precedente, `x` non poteva che valere $0 + 1 = 1$ (non può essere stato dimezzato perché allora sarebbe stato 0 già prima). All'iterazione ancora precedente, `x` valeva $2 \cdot 1 = 2$. Proseguendo, prima ancora `x` poteva valere o $2 + 1 = 3$, oppure $2 \cdot 2 = 4$: siccome vogliamo il minimo, conviene scegliere la prima opzione. Subito prima `x` valeva per forza $2 \cdot 3 = 6$ (non poteva valere 4, altrimenti sarebbe stato dimezzato). Infine, all'iterazione precedente (siamo arrivati a 5) `x` valeva $6 + 1 = 7$ oppure $2 \cdot 6 = 12$.

Selezioni 2021-22, Esercizio 6. La risposta è 8.

La funzione è ricorsiva. Quando viene chiamata `f(13)`, siamo nel terzo caso (13 non è né minore né uguale a 10). Allora viene restituito `f(13 - 2) - 1`, quindi dobbiamo capire cosa fa `f(11)`. Ricadiamo ancora nel terzo caso, quindi viene restituito `f(9) - 1`, ovvero la chiamata iniziale restituisce `f(9) - 2`. Quando viene chiamata `f(9)`, siamo nel primo caso ($9 < 10$), e viene restituito `f(9 + 1) + 3`, che, per la chiamata iniziale, dà `f(10) + 1`. Dato che `f(10)` restituisce 7, la risposta è $7 + 1 = 8$.